

Rapport PAI2D

Anais Azouaoui et Caroline Zouloumian

Avril 2026

Table des matières

1	Introduction	3
2	Etat de l'art	3
3	Contribution	3
4	Méthodologie	5
4.1	Préliminaire	5
4.1.1	Architecture globale	5
4.1.2	Bibliothèque d'objets	5
4.2	Traitement des entrées : génération de la scène	5
4.2.1	Pipeline texte	5
4.2.2	Pipeline image	10
4.3	Construction de la scène	13
4.3.1	Orientations	13
4.3.2	Dimensions	13
4.3.3	Hierarchie des relations	13
4.3.4	Placement des objets dans la scène	14
4.4	Boucle de validation	14
4.4.1	Validation physique	15
4.4.2	Sélection de l'angle de caméra (mode image)	15
4.4.3	Validation sémantique	15
5	Résultats	16
5.1	Métriques et protocole expérimental	16
5.2	Évaluation du pipeline texte	16
5.2.1	Reconnaissance d'objets	17
5.2.2	Placement : relations et orientations	17
5.2.3	Latence	18
5.3	Évaluation du pipeline image	19
5.3.1	Protocole	19
5.3.2	Résultats V3 vs V3.1	20
5.4	Évaluation de la boucle de validation physique et sémantique	21
5.4.1	Validation physique et effet sur les relations	21
5.4.2	Convergence de la boucle	22
5.4.3	Synthèse	22
5.5	Validation sémantique du système par étude utilisateur	23
5.6	Synthèse comparative	24
6	Discussion	24
7	Conclusion	25

1	Annexe 1 : Manuel d'utilisation	27
1.1	Vue d'ensemble de l'interface	27
1.2	Description des éléments	27
1.2.1	Barre de sélection des modèles	27
1.2.2	Panneau de chat	27
1.2.3	Aperçu 3D de la scène	28
1.2.4	Validation de la scène	28
1.2.5	Simulation Genesis	28
1.2.6	Téléchargement de la scène	28
1.3	Workflow typique	29
2	Annexe 2 : Cahier des charges	30
2.1	Introduction	30
2.2	Objectifs	30
2.3	Description du système	30
2.3.1	Module d'interprétation sémantique (LLM / VLM)	30
2.3.2	Module de génération de scène Genesis	31
2.3.3	Boucle de validation et correction	31
2.3.4	Interface utilisateur	32
2.3.5	Architecture globale	32
2.4	Contraintes	32
2.5	Évaluation et critère d'acceptation	32
2.6	Planning	32
2.6.1	Deadlines générales du projet AI2D	32
2.6.2	Notre organisation	32

1 Introduction

La création manuelle de scènes de simulation pour la robotique est coûteuse en temps et difficilement reproductible à grande échelle. Or, la diversité des environnements d’entraînement est un facteur déterminant pour la généralisation des politiques d’apprentissage par renforcement. La génération procédurale de scènes constitue une réponse à ce problème, mais elle reste encore largement cantonnée à des approches heuristiques ou à des environnements simplifiés.

Genesis[1] est un simulateur physique pour la robotique, qui supporte la génération de scènes 3D à partir de descriptions URDF et de maillages OBJ/PLY, avec un moteur physique capable de gérer les contacts, collisions et dynamique des corps rigides. Ce projet propose une couche de génération automatique au-dessus de Genesis : à partir d’un prompt en langage naturel ou d’une photographie d’un environnement réel, le système produit une scène JSON décrivant précisément les objets, leurs orientations et leurs positions tridimensionnelles, prête à être chargée et simulée dans Genesis.

L’approche s’appuie sur plusieurs variantes de pipeline combinant des modèles de langage (LLM), des modèles de vision (VLM) et une correspondance par plongement sémantique, avec une boucle de validation itérative qui corrige les violations de contraintes physiques détectées en simulation. La problématique de recherche est la suivante : *comment construire un pipeline automatique qui produit, à partir d’un prompt textuel ou d’une image, de manière locale et efficace, une scène tridimensionnelle physiquement cohérente et directement exploitable par un simulateur robotique.*

Le rapport est organisé comme suit. La section 2 situe notre travail par rapport à la littérature existante. La section 3 formalise nos contributions, qui sont ensuite détaillées dans la méthodologie (4). La section 5 présente les évaluations menées ; nous incluons notamment des données récoltées sur un robot réel et auprès de personnes extérieures, et la section 6 en discute les limites et les perspectives.

Ce projet a été réalisé dans le cadre de l’UE PAI2D du Master 1 AI2D à Sorbonne Université, sous l’encadrement du Docteur Emiland Garrabe et du Professeur Stéphane Doncieux de l’Institut des Systèmes Intelligents et de Robotique de Sorbonne Université (ISIR).

2 Etat de l’art

Plusieurs travaux explorent la génération automatique de scènes pour la robotique. *ProcTHOR* [2] génère, de manière procédurale, des milliers d’intérieurs de maison pour l’embodied AI, démontrant qu’un passage à l’échelle est possible, mais il n’inclut pas d’entrée utilisateur. *Holodeck* [3] introduit le pilotage de l’environnement AI2-THOR par un LLM, ouvrant la voie au contrôle sémantique de la génération. *RoboGen* [4] combine la génération de tâches et de scènes pour produire des données d’entraînement pour la robotique. Ces travaux illustrent l’intérêt pour la génération automatisée de scènes de simulation et constituent la littérature de référence dans laquelle s’inscrit ce projet.

Sur le plan des modèles, deux aspects de la littérature ont guidé nos choix d’architecture. D’une part, le raisonnement spatial des VLM reste limité : *SpatialVLM* [5] montre que même un VLM entraîné explicitement à raisonner spatialement produit des estimations imprécises, sans données absolues de distance. D’autre part, les modèles de langue locaux, et parfois même les plus grands, peinent à inférer des relations spatiales cohérentes à partir d’un simple prompt textuel.

Ce projet s’inscrit enfin dans la thématique du *sim-to-real gap* [6, 7], qui désigne l’écart de performance entre un robot entraîné en simulation et son comportement en conditions réelles. La *domain randomization* a montré que diversifier les conditions de simulation améliore la généralisation : notre travail explore, à une échelle modeste, la diversification sémantique des scènes comme un levier complémentaire.

Travail	Entrée	Simulateur	Valid. physique	Multimodal
ProcTHOR [2]	—	AI2-THOR	non	non
Holodeck [3]	texte	AI2-THOR	non	non
RoboGen [4]	tâche	divers	partielle	non
Notre système	texte + image	Genesis	oui	oui

TABLE 1 – Positionnement par rapport aux travaux les plus directement comparables.

3 Contribution

Notre travail apporte quatre contributions principales.

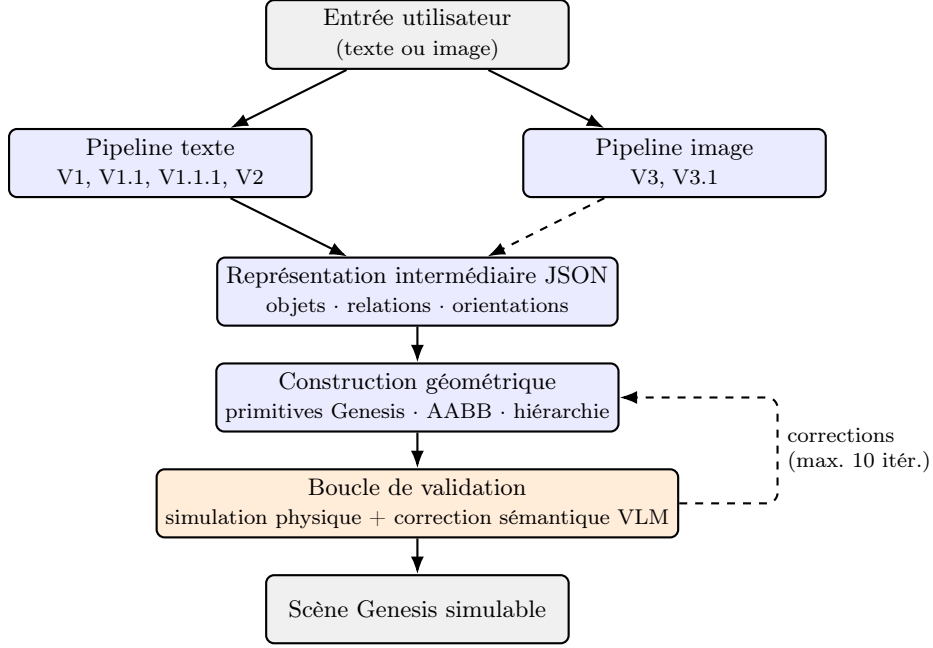


FIGURE 1 – Architecture du système proposé. L’entrée multimodale est traitée par l’un des deux sous-pipelines. La construction géométrique calcule les positions et les orientations à partir des boîtes englobantes extraites par Genesis. La boucle de validation simule la scène et soumet des corrections relationnelles au modèle vision-langage jusqu’à dix itérations.

(1) Génération multimodale de scènes compatibles Genesis Nous proposons un pipeline unifié de génération de scènes 3D à partir de texte ou d’image, produisant directement des scènes exécutables dans Genesis. Contrairement aux approches reposant principalement sur les capacités spatiales des LLM, les dimensions géométriques sont systématiquement mesurées dans le simulateur à partir des maillages effectivement utilisés et d’une représentation JSON intermédiaire, ce qui garantit, en principe, la cohérence physique des scènes générées.

(2) Validation physique-sémantique contrainte Nous introduisons une boucle de validation combinant simulation physique et correction par modèle vision-langage. Les corrections sont limitées à des relations symboliques (`on`, `left_of`, etc.) plutôt qu’à des coordonnées numériques, afin d’éviter les hallucinations spatiales et de préserver une cohérence géométrique.

(3) Fine-tuning QLoRA de Phi-3-mini pour l’extraction structurée Nous remplaçons une architecture multi-appels LLM par un unique modèle Phi-3-mini[8] affiné via QLoRA[9]. L’entraînement repose sur un dataset synthétique généré procéduralement et validé automatiquement par validation croisée entre différents modèles.

(4) Étude d’ablation progressive du pipeline Le développement de cinq versions successives du pipeline (V1 à V3.1) est exploité comme une étude d’ablation. Chaque version modifie un composant unique par rapport à la précédente : positions calculées par LLM *versus* calculées par primitives géométriques, correspondance catalogue exacte *versus* par embeddings multilingues, multi-appels au LLM *versus* appel unique à Phi-3 affiné, modalité texte *versus* modalité image, détection grossière *versus* raffinée par GroundingDINO[10]. Cette structure permet d’isoler l’effet de chaque décision sur l’architecture par rapport à un même protocole d’évaluation.

Il y a plusieurs extensions possibles du système que nous avons gardées hors du périmètre de ce travail. Nous ne traitons pas les objets articulés (tiroirs ouverts, portes battantes, etc), ni les relations trop complexes pour le moment. Enfin, l’évaluation *downstream* par entraînement d’une politique d’apprentissage par renforcement sur les scènes générées dépasse le cadre temporel du projet et est discutée en perspective dans la section relative aux travaux futurs.

4 Méthodologie

Cette section décrit le système de bout en bout, depuis l’entrée utilisateur jusqu’à la scène simulable. La section 4.1 fixe les conventions et présente le catalogue d’objets, commun à toutes les variantes. La section 4.2 détaille la génération de la représentation intermédiaire à partir d’un prompt texte ou d’une image, version par version. La section 4.3 décrit la construction géométrique effective de la scène, et la section 4.4 la boucle de validation itérative qui détecte et corrige les violations résiduelles.

4.1 Préliminaire

4.1.1 Architecture globale

La figure 1 donne une vue d’ensemble du pipeline : l’entrée utilisateur, texte ou image, est traitée par l’un des deux sous-pipelines, qui produisent une représentation intermédiaire au format JSON. Celle-ci alimente le module de construction, et la sortie est soumise à la boucle de validation. Dans la suite du rapport, nous désignons les cinq variantes successives du pipeline par V1, V1.1, V1.1.1, V2, V3 et V3.1. Cette numérotation reflète l’historique du développement : V1 à V2 forment la chaîne d’ablation du pipeline de texte, dans laquelle chaque version modifie un seul composant par rapport à la précédente, tandis que V3 et V3.1 traitent l’entrée d’image. La version V2 est la version par défaut du module texte et V3.1 celle du module image.

4.1.2 Bibliothèque d’objets

La bibliothèque d’objets, ou *catalogue*, regroupe les 28 objets que le système peut instancier dans une scène, chacun sous la forme d’un dossier contenant un fichier URDF ou OBJ. Cette bibliothèque est extensible ; nous avons contrôlé le surapprentissage du système. Les objets manipulables, tels que la banane, le mug, les ciseaux ou la boîte de gélatine, proviennent du jeu YCB [11] ; les éléments de mobilier, tels que le réfrigérateur, le lave-linge ou la porte, proviennent de PartNet-Mobility [12]. Après plusieurs itérations, nous avons constaté que la bibliothèque fonctionnait le mieux avec la version 0.3.3 de Genesis.

4.2 Traitement des entrées : génération de la scène

4.2.1 Pipeline texte

Le pipeline “texte” a évolué en quatre versions successives, chacune corrigeant un défaut identifié dans la précédente. La présente sous-section décrit les trois premières, V1, V1.1 et V1.1.1, qui constituent les itérations préparatoires ; la version V2, qui constitue la contribution principale de ce travail et la version par défaut du système, fait l’objet de la sous-section suivante.

V1 — version d’ancrage, un unique appel LLM. La première version, et la plus intuitive, du pipeline confie au LLM l’intégralité de la tâche en un seul appel : à partir du prompt utilisateur et du catalogue complet fourni en contexte, le modèle retourne directement une liste d’objets accompagnés de leur identifiant URDF, de leur facteur d’échelle, de leurs quaternions d’orientation et de leurs positions tridimensionnelles. Cette approche est la plus simple à implanter et la plus rapide en inférence via l’API GPT (environ 7 s par scène), mais elle présente deux limites. D’une part, le modèle doit rendre des URDF exacts du catalogue, ce qui provoque des hallucinations de labels et nécessite trois tentatives de retry et une validation pour obtenir des résultats concluants. D’autre part, les modèles de langues ne sont pas optimisés pour la compréhension sémantique de la 3D et sont peu performants avec les chiffres. Les coordonnées produites sont souvent peu précises, voire complètement faussées. La fonction `modify_scene()` (qui permet, par un second appel, de modifier une scène déjà générée, fonctionnalité peu développée pour le moment) de V1 reste néanmoins conservée dans l’interface utilisateur pour effectuer des modifications ponctuelles.

V1.1 — séparation extraction / placement. V1.1 introduit la séparation structurelle qui sera conservée dans toutes les versions ultérieures : le modèle de langue n’est plus chargé de calculer les positions, mais uniquement d’extraire une description symbolique de la scène, comme l’illustre la figure 2. Cette description intermédiaire est ensuite interprétée par le module de construction, détaillé dans la section 4.3, afin de placer effectivement les objets. L’extraction est elle-même décomposée en quatre appels séquentiels : reconnaissance des objets et association aux identifiants URDF du catalogue, validation sémantique des objets associés via un second appel à `validate_matches()`, extraction des relations

spatiales (et de distances optionnelles) en mètres, puis extraction des orientations symboliques. Cette restructuration porte le F1 de reconnaissance à 0,828 avec GPT-4o, à 0,518 avec Llama 3.1 [13], et permet d’obtenir un F1 de relations mesuré à 0,563 avec GPT-4o, à 0,301 avec Llama. Deux limites subsistent toutefois et motivent la version suivante. Le modèle reste contraint de produire les identifiants exacts du catalogue, ce qui maintient le risque d’hallucination, et la séquence de quatre appels engendre une latence cumulée de l’ordre de 9 s avec GPT-4o et de 140 s avec Llama 3.1. La séquentialité induit également un risque d’incohérence inter-appels lorsque les identifiants renvoyés à la première étape ne se retrouvent pas exactement dans les triplets de relations produits à la troisième étape.

```
{
  "objects": ["lave_linge", "mug", "banane"],
  "relations": [
    {"type": "on", "subject": "mug", "object": "lave_linge"},
    {"type": "right_of", "subject": "banane", "object": "mug"}
  ],
  "orientations": [{"object": "mug", "type": "upside_down"}]
}
```

FIGURE 2 – Représentation intermédiaire produite par V1.1, V1.1.1 et V2 pour le prompt « un lave-linge avec un mug à l’envers sur celle ci et une banane à droite du mug ». Les positions tridimensionnelles et les quaternions d’orientation ne figurent pas à ce stade : ils sont calculés par le module de construction géométrique (section 4.3) à partir des relations symboliques et des dimensions extraites par Genesis.

V1.1.1 — matching catalogue via embeddings. La principale source d’erreurs de V1.1 étant la contrainte imposée au modèle de langue de connaître les identifiants exacts du catalogue et la saturation de son contexte, conduisant à un passage à l’échelle peu performant, V1.1.1 sépare la compréhension linguistique du matching au catalogue. Le modèle se contente d’extraire des labels libres des objets dans la langue du prompt (par exemple, pour un prompt “je veux un mug sur une table”, le LLM extraira les mots “mug” et “table” sans se soucier de leur existence dans le catalogue) et la mise en correspondance avec les identifiants URDF (exp : “table” → “001_table”) du catalogue est déléguée à un modèle d’embedding multilingue, *multilingual-e5-base* [14, 15], qui convertira ces derniers en vecteurs numériques. La similarité cosinus entre les vecteurs-objet et les entrées du catalogue est comparée à un seuil empirique fixé à 0,55. Lorsque celle-ci est plus élevée, il y a match. En outre, cette séparation permet désormais de reconnaître des synonymes courants (comme *frigo* et *réfrigérateur*, *lave-linge* et *washing machine*) et de gérer les fautes d’orthographe ainsi que le bilinguisme français-anglais.

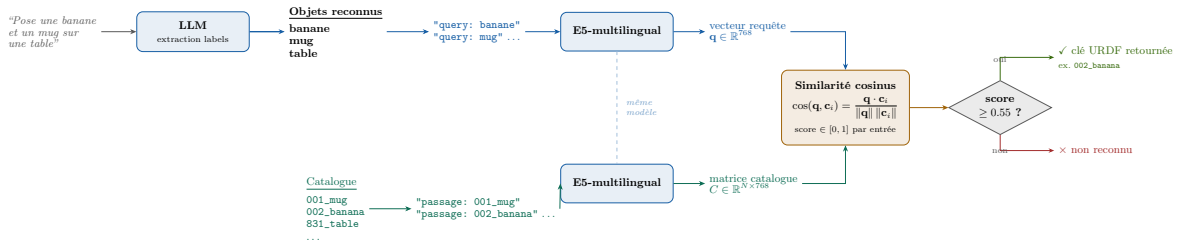


FIGURE 3 – Pipeline V1.1.1 de reconnaissance d’objets par embeddings.

Toutefois, malgré cette amélioration du matching sémantique, la chaîne reste encore composée d’appels séquentiels, donc prône à l’hallucination, et coûteuse, particulièrement sur des machines peu performantes. Ce sont ces deux principaux défauts qui ont motivé la création de notre dernière version à jour, V2.

V2 — pipeline affiné, appel local unique. La version V2 constitue la version par défaut du système et regroupe les trois appels successifs au LLM des versions précédentes (extraction des labels, des relations et des orientations) en une seule requête à un modèle Phi-3-mini affiné localement, baptisé **phi3-scene**. Le motif principal de ce regroupement est double : réduire la latence cumulée et la surface d’erreur de la chaîne séquentielle, et supprimer la dépendance à une API externe pour rendre l’inférence reproductible et rapide après entraînement (et gratuite!). La sortie est un unique objet JSON contenant les trois champs **objets**, **relations** et **orientations**, exploité ensuite par le module de résolution catalogue

par embedding (même système introduit par V1.1.1) puis par la construction géométrique décrite en section 4.3.

Modèle de base. Nous partons de Phi-3-mini-4k-instruct [8], un *small language model* de 3,82 milliards de paramètres (soit 2 fois moins que Llama3.1) publié par Microsoft, dans la variante `unsloth/Phi-3-mini-4k-instruct-bnb-4bit` préquantifiée en 4-bit.

Construction du dataset d’entraînement. Aucune source publique ne fournit le type de données dans le format propre à notre pipeline. Nous avons donc construit notre propre dataset de manière synthétique, en mobilisant l’API GPT-4.1-mini.

On pourrait, à juste titre, trouver cette démarche problématique : si nous reprochons aux LLM leurs hallucinations au point d’en faire le motif central de notre architecture, il est difficile de leur confier en même temps la fabrication de nos données d’entraînement. Voici comment nous avons rendu cette approche fiable malgré tout.

La solution consiste à inverser le sens habituel de la génération. L’approche naïve partirait d’un corpus de phrases et demanderait à un LLM d’en produire l’annotation JSON, elle cumulerait deux sources d’erreur, dans la phrase elle-même et dans son annotation. Nous procédons dans l’autre sens : la cible JSON est produite *d’abord* sans appel au LLM via le module `spec_generator.py`, puis la phrase naturelle est générée *ensuite* à partir de cette cible. La structure de la sortie attendue est ainsi fixée par construction, plus par génération, et le LLM n’intervient que sur la partie où il est le plus fiable, à savoir la paraphrase d’une spécification déjà figée.

Le module `spec_generator.py` tire procéduralement une cible JSON à partir d’un pool de 83 objets français et 20 objets anglais (qui ne figurent pas tous dans notre catalogue d’objets, pour contrôler l’overfitting), de nos 8 relations spatiales (`left_of`, `right_of`, etc.) et de nos 8 orientations canoniques (`tip_left`, `tip_right`, etc.) définies plus en détail en section 4.3. Chaque tirage suit l’une de onze configurations pondérées, chacune ciblant une difficulté linguistique que nous avons rencontrée lors des tests avec d’autres LLMs : scènes simples (`basic`, 19 %), empilements (`stacking`, 11 %), surfaces communes et alignements (11 % chacun), distances explicites, scènes denses à plusieurs objets, phrases coréférentielles du type « *cette tasse* », instances multiples du même objet, phrases longues, prompts en anglais (4 %) et scènes avec orientations explicites. Cette pondération est notre principal levier de contrôle sur ce que le modèle apprend. Augmenter la part de `coreference` l’expose davantage aux pronoms démonstratifs, augmenter `multi_instance` le force à respecter la convention d’instanciation `banane`, `banane_2` lorsqu’il y a plusieurs bananes par exemple, et ainsi de suite. Certains objets sont en outre tenus à l’écart du pool d’entraînement pour constituer un *held-out* sur lequel évaluer la capacité de généralisation en dehors de celui-ci.

Une fois la cible JSON tirée, `phrase_generator.py` en demande la traduction en langage naturel à GPT-4.1-mini. Reste néanmoins un risque résiduel : le modèle peut produire une phrase qui s’écarte de la cible, oublier, inventer un objet, ou inverser une relation, même si cela est une forme d’hallucination que nous avons remarquée

phi3-scene v1 (100 exemples)	
Métrique	Score
JSON parse	99.0%
F1 objets	85.3%
Relation F1	59.0%
Relation exact match	34.0%
Orientation exact	68.0%
F1 held-out (gén.)	87.1% (37 ex)

phi3-scene v2 (100 exemples)	
Métrique	Score
JSON parse	98.0%
F1 objets	93.7%
Relation F1	69.0%
Relation exact match	44.0%
Orientation exact	78.0%
F1 held-out (gén.)	94.3% (37 ex)

phi3-scene v3 (100 exemples)	
Métrique	Score
JSON parse	99.0%
F1 objets	98.4%
Relation F1	77.0%
Relation exact match	53.0%
Orientation exact	86.0%
F1 held-out (gén.)	98.8% (37 ex)

Évaluation sur 100 prompts validés manuellement. F1 held-out sur 37 exemples hors distribution. Vert $\geq 95\%$, bleu $\geq 80\%$, orange $< 80\%$.

TABLE 2 – Résultats d’évaluation des trois versions de phi3-scene.

plus rare. Pour s'en prémunir, un second appel reçoit la phrase et le JSON, et doit répondre à la question question : "cette phrase dit-elle exactement ce que le JSON décrit ?" Si la réponse est négative, la paire est rejetée et n'entre pas dans le dataset. La figure 5 illustre ce flux sur un exemple concret. Le dataset final compte 3 001 paires d'entraînement, 353 de validation et 177 de test, soit 3 531 exemples validés. Nous avons également réalisé une multitudes d'opérations de nettoyage pour minimiser le biais du dataset, et avons obtenu le tableau de métriques suivant². Plusieurs itérations de constructions de dataset et de choix de specs ont mené à la version finale de phi3-scene. Trois versions successives ont été évaluées sur **100 prompts** générés automatiquement et validés manuellement. Chaque version intègre des corrections de données d'entraînement et un ajustement du format de sortie attendu (voir Table 2).

Les métriques mesurées sont :

- **JSON parse** : taux de réponses parsables sans erreur.
- **F1 objets** : F1 sur la liste des objets extraits.
- **Relation F1** : F1 sur les relations spatiales.
- **Relation exact match** : relations *exactement* correctes (type + arguments).
- **Orientation exact match** : orientation des objets correcte à ± 1 classe.
- **F1 held-out** : F1 objets sur 37 exemples hors distribution (généralisation).

Les principales difficultés restantes concernent les relations spatiales complexes (Relation exact match $\approx 53\%$) et l'orientation. Augmenter la quantité de données avec les moyens suffisants pourrait pallier ce problème.

Configuration QLoRA. Réentraîner un grand modèle de langue *from scratch* demanderait des semaines de calcul et des dizaines de GPU, ce qui est hors de portée à notre stade. La technique QLoRA contourne ce problème en n'entraînant qu'une infime fraction du modèle. En outre, nous gelons tous les poids existants et y greffons de petites couches supplémentaires, appelées adaptateurs LoRA [16]. Pour réduire encore la mémoire consommée, le modèle de base est chargé en précision 4-bit, ce qui divise grossièrement la taille en mémoire par quatre, au prix d'une légère perte de précision compensée par le fine-tuning.

Ces adaptateurs sont insérés dans les sept couches responsables de l'attention et du traitement de l'information au sein du transformeur : `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj` et `down_proj`. Les réglages retenus (rang $r = 16$, facteur d'échelle $\alpha = 32$, pas de dropout ni de biais) contrôlent la capacité expressive de ces adaptateurs : un rang plus élevé autorise des adaptations plus complexes, au prix d'un coût mémoire accru. L'entraînement est supervisé (SFT) : nous montrons au modèle des exemples de dialogues au format propre à Phi-3 — structuré en tours `system` / `user` / `assistant` / `end` — et il apprend à imiter ces réponses.

Hyperparamètres et infrastructure. La taille de batch effective est de 16 (4 par device avec 4 étapes d'accumulation), pour 3 epochs, un taux d'apprentissage de $2 \cdot 10^{-4}$ avec un *scheduler* cosinus, un *warmup* de 50 pas, un optimiseur AdamW 8-bit et une graine fixée à 42. L'infrastructure est un GPU Tesla T4 via Google Colab, utilisé via la bibliothèque Unsloth [17] qui permet un entraînement LoRA 4-bit plus économe en mémoire. La sortie des courbes de *loss* est présentée ci dessous.

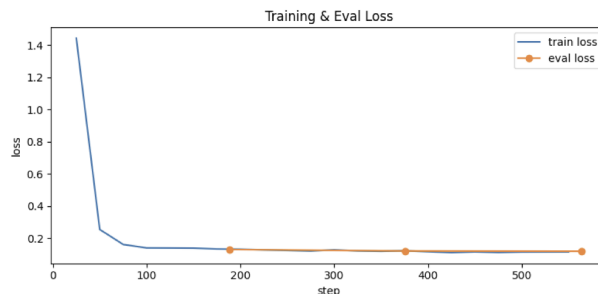


FIGURE 4 – Courbe de loss : La loss d'entraînement chute très rapidement puis se stabilise autour de 0.12–0.14, ce qui indique que le modèle apprend efficacement dès les premières étapes. La courbe d'évaluation suit de près la train loss sans divergence notable, ce qui suggère peu ou pas d'overfitting. Le fine-tuning a convergé correctement, et continuer beaucoup plus longtemps apporterait probablement des gains marginaux.

L'entraînement a duré 3 epochs, le modèle a donc vu l'intégralité des données d'entraînement trois fois de suite. À chaque étape, 16 exemples sont traités simultanément (batch effectif de 16). Le taux d'apprentissage est fixé à $2 \cdot 10^{-4}$.

Intégration en inférence. À l'issue de l'entraînement, les adaptateurs LoRA sont fusionnés avec le modèle de base pour produire un modèle autonome complet. Il est ensuite converti au format GGUF et quantifié en Q4_K_M via `llama.cpp` : une technique de compression qui réduit encore la taille du modèle en approximant ses poids, rendant l'espace mémoire du modèle final seulement 2,32 Go, contre 4,9 Go pour LLaMA 3.1 dans une précision comparable, et déployable sur n'importe quelle machine locale sans GPU dédié.

Le modèle est servi via Ollama, piloté par un `Modelfile` (voir `training/Modelfile`) qui joue le rôle de notice d'instruction : il définit le prompt système (ce que le modèle « sait » de son rôle), lui assigne une tâche unique, et fixe la température à 0. La température contrôle le caractère aléatoire des réponses : à 0, le modèle choisit toujours le token le plus probable, ce qui garantit qu'un même prompt produira toujours la même sortie, un comportement indispensable pour notre pipeline. Finalement, la résolution catalogue ne change pas par rapport à V1.1.1, mais utilise un modèle d'embeddings différent, avec un seuil de similarité cosinus à 0,40, plus permissif que le 0,55 de V1.1.1 pour compenser la nature plus libre des labels produits par le modèle affiné.

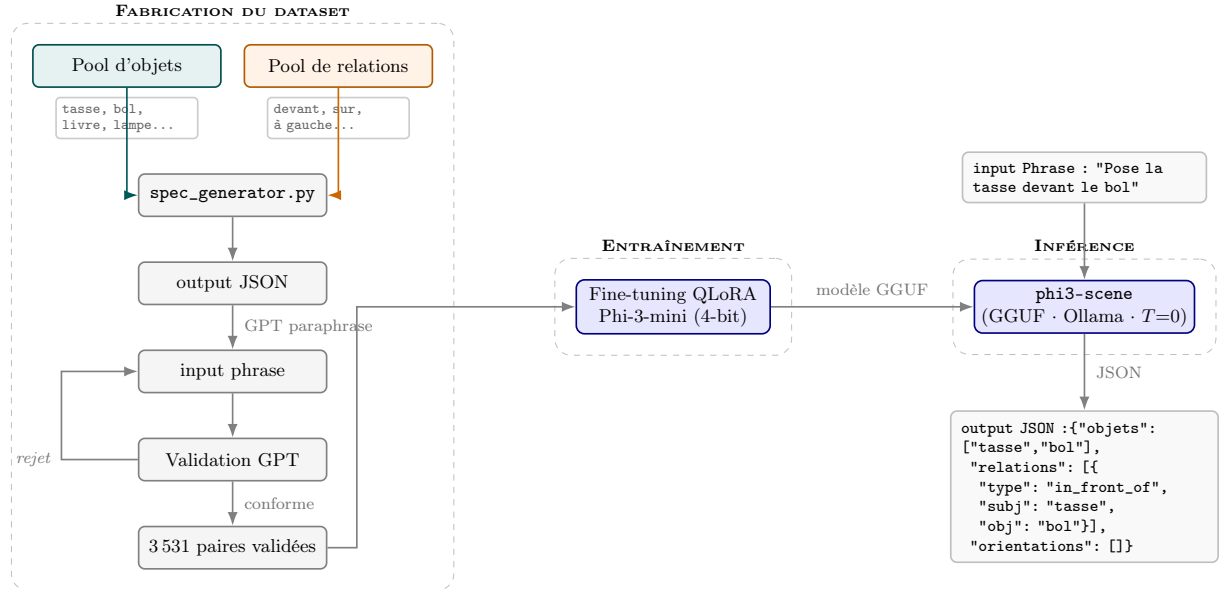


FIGURE 5 – Pipeline V2 : fabrication synthétique et validation du dataset, fine-tuning QLoRA sur Phi-3-mini, et inférence locale avec `phi3-scene`, illustrée par un exemple de prompt et sa sortie JSON.

Ainsi, avec cette structure, notre système V2 améliore la reconnaissance des relations et des objets, mais aussi considérablement le temps d'inférence, qui était l'un de nos principaux facteurs de ralentissement lors de l'exécution de ce projet. Nous rapportons ici des métriques obtenues sur le test set de 177 exemples générés, incluant des objets jamais vus à l'entraînement pour évaluer la généralisation.

Métrique	Score
JSON parse OK	99,0%
F1 objets	98,4%
Root accuracy	99,0%
Relation F1	77,0%
Relation exact match	53,0%
Orientation exact match	86,0%
F1 held-out (généralisation, 57 ex.)	98,8%

TABLE 3 – Résultats sur 177 exemples.

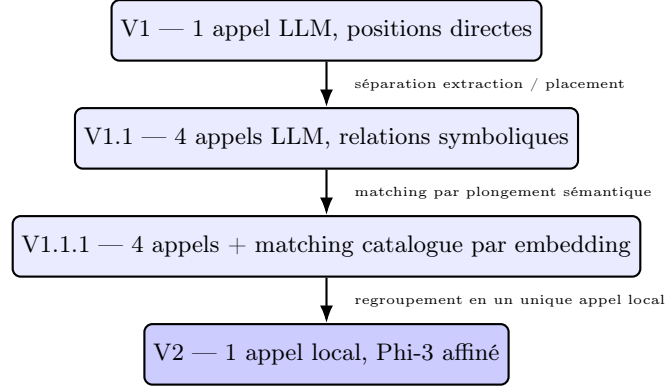


FIGURE 6 – Évolution chronologique du pipeline texte. Chaque flèche identifie le changement architectural qui motive la version suivante.

4.2.2 Pipeline image

L’extension du pipeline à la modalité image se heurte à une limite bien des modèles vision-langage actuels : le raisonnement spatial tridimensionnel à partir d’une image plane reste fragile, même pour des modèles entraînés explicitement à cette tâche. *SpatialVLM* [5] montre par exemple que les estimations métriques produites par un VLM dédié restent imprécises en l’absence d’information de profondeur, et les approches plus récentes capables d’opérer directement dans l’espace 3D demeurent hors de portée de nos contraintes matérielles. Nous proposons donc deux versions successives, V3 et V3.1, qui répartissent progressivement les responsabilités entre le VLM et des modules dédiés.

V3 — détection directe par VLM. La version V3 est l’analogie image de V1 : un unique appel au VLM prend en charge l’intégralité de l’extraction. L’image (ou les images, dans le cas multi-vues) est encodée en base64 avec le paramètre `detail: high`, puis transmise au modèle avec un prompt structuré demandant la liste des objets reconnus, leurs positions estimées et leurs orientations canoniques parmi les huit *turns* définis en section 4.3. Les paramètres d’appel sont `temperature=0` et `max_tokens=2048`. Le filtrage par catalogue n’intervient qu’en aval : les labels retournés par le VLM sont confrontés aux identifiants URDF connus et les objets hallucinés sans correspondance sont rejetés. Cette architecture transfère donc au modèle la totalité du raisonnement spatial, exactement comme V1 le faisait pour la modalité texte. Le résultat, mesuré section 5.3, est analogue, les positions retournées sont approximatives et les boîtes englobantes implicites trop vagues pour être directement exploitables. Ce constat motive la version V3.1.

V3.1 — raffinement par GroundingDINO et projection métrique. La version V3.1 reprend le principe architectural de V1.1.1 : confiner le VLM à ce qu’il fait le mieux (reconnaissance et raisonnement relationnel symbolique) et déléguer l’extraction géométrique précise à un module spécialisé. Concrètement, un détecteur d’objets zero-shot, *GroundingDINO* [10], est intercalé entre le VLM et le module de construction géométrique. Le pipeline procède en cinq étapes (figure ??) :

1. **Extraction sémantique par VLM.** GPT-4o reçoit l’image et retourne, pour chaque objet détecté, un label français, un label anglais (pour la requête à GroundingDINO), une position normalisée $pos_norm \in [0, 1]^2$ exprimée en coordonnées image, une orientation parmi les huit *turns* canoniques, ainsi que le type de vue ($view_type \in \{\text{frontal}, \text{top_down}\}$) et la liste des relations spatiales perçues. Les paramètres d’appel sont resserrés par rapport à V3 : `max_tokens=1024` et `response_format=json_object`, ce qui contraint la sortie à un objet JSON parsable. Si plusieurs vues d’une même scène sont fournies, elles sont transmises simultanément et le VLM est invité à les fusionner.
2. **Raffinement des contours par GroundingDINO.** Le modèle `IDEA-Research/grounding-dino-tiny` reçoit l’image et la liste des labels anglais extraits par GPT-4o, puis retourne les boîtes englobantes correspondantes (seuils $box_threshold = 0,3$ et $text_threshold = 0,25$). La fonction `assign_contours` associe ensuite chaque boîte à l’instance d’objet correcte : pour un label unique, l’association est directe ; en cas de doublons (par exemple deux *banane* retournant deux boîtes *banana*), la désambiguïsation se fait par distance euclidienne entre le centre de chaque boîte et le

pos_norm fourni par le VLM. Une fois l'association faite, le pos_norm initial est remplacé par le centre de la boîte GroundingDINO, plus précis. Si un objet n'est pas détecté par GroundingDINO, le pos_norm du VLM est conservé en repli.

3. **Résolution catalogue par embeddings multilingues.** Cette étape réutilise sans modification le module introduit en V1.1.1 : les labels libres (français ou anglais) sont projetés dans l'espace d'embeddings *multilingual-e5-base* [14] et appariés aux identifiants URDF du catalogue par similarité cosinus avec seuil 0,55.
4. **Projection des coordonnées image vers le monde 3D.** Avant la projection, le module `get_genesis_dimensions` charge chaque mesh sélectionné dans le moteur Genesis, applique la rotation associée et mesure la boîte englobante alignée aux axes (AABB). On obtient ainsi des dimensions 3D $[d_x, d_y, d_z]$ fiables après rotation, dont est dérivée la hauteur de pose au sol $z = d_z/2$. Le passage du repère image normalisé (img_x, img_y) au repère monde dépend ensuite de la disponibilité d'une carte de profondeur et du *view_type*.

Sans carte de profondeur (pos_norm_to_world), aucune information métrique n'est disponible par pixel. On se rabat alors sur une projection orthographique linéaire qui suppose la scène centrée devant la caméra et de faible perspective. Concrètement, on définit arbitrairement une zone de travail rectangulaire devant la caméra, de largeur $W_f = 1,5$ m (étendue latérale) et de profondeur $D_f = 2,0$ m, dans laquelle les coordonnées image sont ré-étalées linéairement.

En vue frontale, la caméra regarde la scène horizontalement : l'axe horizontal de l'image correspond donc naturellement au déplacement latéral de l'objet, tandis que l'axe vertical encode la profondeur, avec la convention que le haut de l'image représente les objets les plus éloignés :

$$world_x = (1 - img_y) \cdot D_f, \quad world_y = (img_x - 0,5) \cdot W_f$$

Le terme $(1 - img_y)$ inverse l'axe vertical de l'image (qui pointe vers le bas) pour qu'il s'aligne avec une profondeur croissante, et le décalage de 0,5 sur img_x recentre l'origine latérale sur l'axe optique.

En vue *top_down*, la caméra regarde la scène à la verticale : les deux axes image se mappent alors directement sur les axes monde, sans changement de nature géométrique :

$$world_x = (img_x - 0,5) \cdot W_f, \quad world_y = (0,5 - img_y) \cdot D_f$$

Cette méthode constitue un repli utilisé lorsque aucune carte de profondeur n'est disponible. Elle reconstruit une profondeur plausible à partir d'une simple hypothèse géométrique, mais en paye le prix : deux objets situés à la même hauteur dans l'image se verront attribuer la même profondeur, indépendamment de leur taille ou de leur distance réelle. Il s'agit donc essentiellement d'un remapping 2D vers une pseudo-3D.

Avec carte de profondeur (depth_to_world), soit fournie sous forme d'un fichier `.npy`, soit prédite par *Depth Anything V2 metric*, l'approche change radicalement : à chaque pixel est désormais associée une distance métrique réelle à la caméra. On peut alors remplacer la projection orthographique par une *back-projection sténopé*, c'est-à-dire l'opération inverse du modèle de caméra pinhole : au lieu de projeter un point 3D sur le plan image, on remonte d'un pixel vers le rayon 3D correspondant, puis on s'arrête à la distance D donnée par la carte de profondeur. Le résultat est une reconstruction métrique qui ne dépend plus de l'hypothèse de scène centrée ni d'une perspective négligeable.

Pour inverser ce modèle, il faut connaître les paramètres intrinsèques de la caméra. Ceux-ci sont reconstruits à partir du *FOV* (champ de vision) estimé par le VLM, typiquement 70° :

$$f_x = \frac{W/2}{\tan(fov_x/2)}, \quad c_x = \frac{W}{2}, \quad c_y = \frac{H}{2}$$

où W et H sont la largeur et la hauteur de l'image en pixels. On suppose des pixels carrés, ce qui donne $f_y = f_x$. La profondeur métrique D est ensuite lue dans la depth map au pixel (u, v) correspondant au centre du contour détecté par GroundingDINO.

En vue *top_down*, la caméra pointe vers le sol et la profondeur D mesure directement la distance verticale entre la caméra et l'objet. Les coordonnées monde dans le plan du sol s'obtiennent alors par la formule classique de back-projection pinhole :

$$world_x = \frac{u - c_x}{f_x} \cdot D, \quad world_y = -\frac{v - c_y}{f_x} \cdot D$$

$(u - c_x)/f_x$ est la *tangente* de l'angle entre le rayon optique central et le rayon passant par le pixel (u, v) . La multiplication par D donne le décalage métrique dans le plan situé à distance D .

En vue frontale, la situation est plus subtile car la caméra n'est pas réellement parallèle au sol, elle est placée à une hauteur h et inclinée d'un angle θ vers le bas (c'est généralement de cette façon que les photos sont prises). La distance D retournée par la depth map est une distance *oblique* le long du rayon optique, et non une distance horizontale dans le plan du sol. Il faut donc projeter ce rayon sur le plan horizontal :

$$world_x = \frac{D - \sin \theta \cdot h}{\cos \theta}, \quad world_y = \frac{u - c_x}{f_x} \cdot D$$

Le numérateur $D - \sin \theta \cdot h$ retire la contribution verticale due à la hauteur de la caméra (la part de D « consommée » par la descente vers le sol), et la division par $\cos \theta$ redresse la distance oblique restante en une distance horizontale sur le plan du sol. La coordonnée latérale $world_y$ suit la même logique que dans le cas *top_down*.

Dans ce second mode, chaque pixel apporte sa propre information de distance et le positionnement en devient plus précis. Toute fois, l'intégration complète de ce pipeline sténopé dans le système n'a pas pu être finalisée dans notre cadre temporel ; et le système, sans contrôle, a tendance à échouer (pour des raisons liées à la librairie de Depth Anything V2 metric que nous n'avons pas encore pu résoudre), d'où la décision de conserver le mode orthographique en repli.

5. **Placement vertical et hiérarchie parent-enfant.** Les coordonnées (x, y) obtenues à l'étape précédente sont complétées par la résolution des relations verticales (**on**, **inside**) via la fonction **processRelations** décrite en section 4.3, qui calcule les z des empilements selon $z_{enfant} = z_{parent} + (d_{z,parent} + d_{z,enfant})/2$ (un parent étant un objet contenant un autre, ou un objet sur-lequel un autre est posé). Pour un enfant posé **on** ou **inside** un parent, la position (x, y) issue de l'étape précédente est aligné avec le contour du parent : les rapports $rel_x, rel_y \in [0, 1]$ sont projetés sur la surface 3D du parent selon *view_type*, avec une marge **SURFACE_MARGIN** = 0,04 m pour éviter les débordements liés au décalage entre l'AABB Genesis et le rendu visuel. Enfin, *uniquement en l'absence de carte de profondeur*, la fonction **nudge_depth_from_relations** applique un décalage de 0,3 m sur l'axe de profondeur pour chaque relation **in_front_of** ou **behind**.

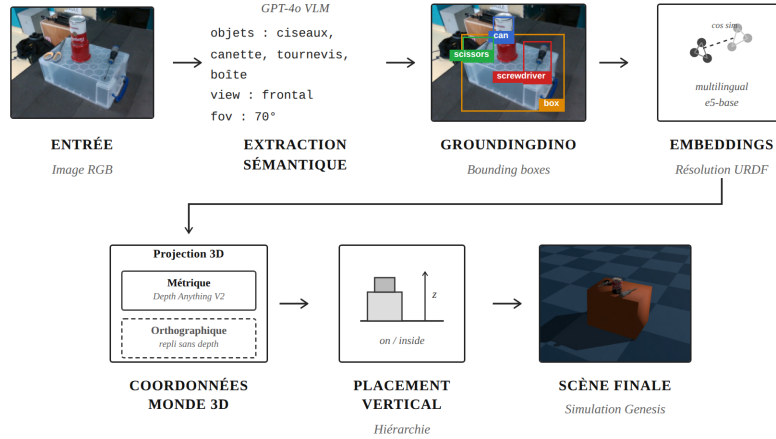


FIGURE 7 – Pipeline complet : extraction sémantique (GPT-4o VLM), détection (GroundingDINO), résolution URDF par embeddings multilingues, projection 3D métrique ou orthographique, placement vertical hiérarchique, et simulation dans Genesis.

Cette architecture hybride conserve une limite liée à l'exploitation encore partielle des cartes de profondeur. Le pipeline avec back-projection sténopé n'est pas pleinement intégré, et la depth map prédite par *Depth Anything V2 metric* reste imparfaite sur les scènes de manipulation robotique, ce qui dégrade la précision métrique en sortie. Cette limite est analysée plus en détail dans la section de discussion.

4.3 Construction de la scène

Afin de transformer des instructions sémantiques en une scène cohérente, nous avons construit une approche par hiérarchie. Celle-ci permet de donner les positions des objets selon leurs relations, après avoir appliqué les rotations nécessaires et établi un ordre cohérent pour le processing des dimensions. Cette partie permet donc la transformation des données sémantiques du JSON, en données simulables physiquement comme la position et les quaternions de chaque objet.

Pour cette section du code, les changements sont appliqués itérativement dans une scène simulée sur Genesis. Cela permet d'éviter les collisions, tout en obtenant les informations les plus précises possibles sur le comportement des objets dans cet environnement de simulation.

La fonction `buildScene` orchestre les différentes étapes de ce processus. Elle crée une scène de simulation Genesis qui va fonctionner en arrière-plan, et y ajoute chaque objet en tant qu'entité. Elle va ensuite, dans cet ordre :

- Appliquer les rotations nécessaires pour obtenir les bonnes orientations des objets.
- Obtenir les dimensions correctes de chaque objet depuis le moteur de simulation.
- Changer la position de chaque objet en fonction de ses relations avec les autres objets.

Cette partie du rapport explique ces différents processus plus en détail.

4.3.1 Orientations

Le premier paramètre des objets à modifier est leur orientation. En effet, l'orientation de l'objet va déterminer ses dimensions perçues : ce qui était de base une dimension peut devenir une autre. Par exemple, en couchant un réfrigérateur sur sa droite (avec la commande `tip_right` dans le fichier JSON), sa hauteur et sa largeur seront échangées l'une avec l'autre dans la scène. Il est nécessaire d'appliquer ces changements pour ne pas fausser le calcul des positions des objets en fonction les uns des autres, et ne pas créer de possibilité de collision.

Les rotations sont exprimées en quaternions car il s'agit du format standard utilisé dans notre simulation sur Genesis. Celles-ci sont donc effectuées par multiplication de quaternions :

$$q_{final} = q_{actuel} \cdot q_{rotation}$$

Les rotations disponibles (`turn_left`, `turn_right`, `turn_around`, `upside_down`, `tip_forward`, `tip_backwards`, `tip_left`, `tip_right`) sont définies par avance, avec les quaternions associés à chaque rotation. Cette nouvelle orientation est ensuite appliquée à l'entité de l'objet sur Genesis, pour qu'elle soit respectée dans la scène.

4.3.2 Dimensions

Afin de pouvoir calculer des positions relatives, nous avons besoin de connaître les dimensions de chaque objet, ainsi que la position de son origine.

En effet, un objet est placé dans Genesis avec les coordonnées de son origine. Cependant, celle-ci n'est pas nécessairement située au centre géométrique de l'objet ; cela dépend des bibliothèques d'objets utilisées. C'est pour cette raison qu'après une première itération de ce code dans laquelle une fonction `getOriginalDimensions` parcourait le fichier URDF pour en extraire les dimensions selon le type de géométrie de l'objet, nous avons choisi d'utiliser directement la simulation Genesis pour avoir une perception exacte des dimensions, de la bounding box, et du placement de l'origine de l'objet.

Nous utilisons la fonction `get_AABB()` de la bibliothèque `genesis-world` pour obtenir la *Axis-Aligned Bounding Box* (AABB) de chaque objet. Contrairement à une simple lecture du fichier `.urdf` de l'objet, cela permet d'obtenir les dimensions après avoir appliqué les rotations demandées. A partir des données de la bounding box, nous pouvons extraire plusieurs informations :

- Les dimensions exactes de l'objet : largeur, profondeur et hauteur.
- La différence entre le point le plus bas de l'objet et le sol, qui nous donne la hauteur de l'origine de l'objet. Cet `offset` est utilisé pour les relations de hauteur (`on` et `under`) lorsqu'elles sont exploitées. Cependant, il est d'abord utilisé ici pour garantir que chaque objet débute avec un `lowest_point` (le point le plus bas de l'objet) soit toujours situé à `1mm` du sol pour chaque objet avant de modifier les positions. Ainsi, aucun objet ne rentre dans ni n'est collé au sol.

4.3.3 Hiérarchie des relations

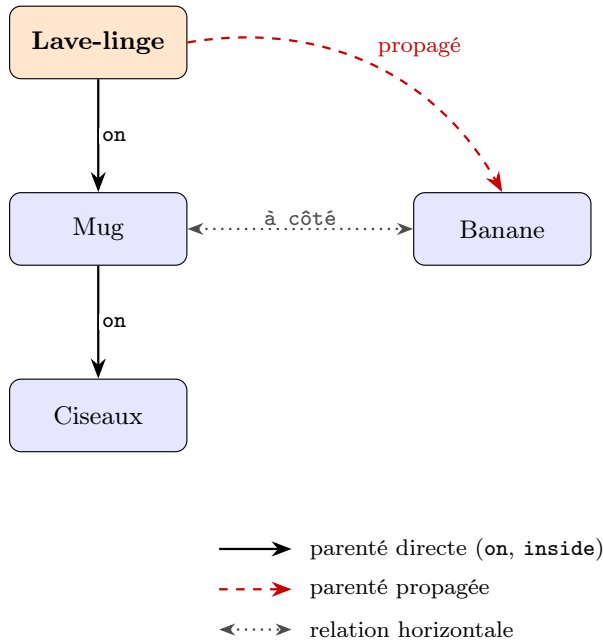


FIGURE 8 – Graphe de scène pour le prompt « *Un mug sur un lave-linge, avec des ciseaux sur ce mug, et une banane à côté de ce mug* ». Les flèches pleines indiquent une parenté directe (on), la flèche rouge pointillée indique une parenté propagée : la banane hérite du parent du mug et devient enfant du lave-linge.

La première version de calcul des positions calculait ces dernières par paires, tour à tour. Un objet *root*, "racine", était établi par le LLM, et était positionné au centre de la scène, sur le sol. Les positions des objets étaient ensuite calculées tour à tour en lien avec un objet déjà placé, en bouclant les relations jusqu'à ce qu'elles aient toutes été utilisées. Nous gardons cette logique pour notre version finale, mais avons raffiné l'approche pour répondre à un problème : le manque de relation de parenté entre les objets. En effet, un objet posé à côté d'un objet qui lui-même était posé sur une table pouvait être placé en dehors des limites de la table dans ce système.

Dans notre approche finale, nous créons un graphe de scène avec la fonction `build_scene_graph`, qui permet l'implémentation d'une hiérarchie de dépendances entre nos entités. Les relations *on* et *inside* sont identifiées comme des liens de parenté directs, avec l'objet au-dessus, ou dans, devenant "enfant" de l'autre. Nous avons ensuite appliqué une propagation itérative des parents. Ainsi, si un mug est placé sur un lave-linge et une banane est placée à côté du mug, la relation de parenté est propagée, et la banane devient également un enfant du lave-linge. Cela permet que chaque contrainte imposée à une entité dans le graphe soit répercutée sur tous ses descendants, ce qui assure une meilleure cohérence spatiale.

4.3.4 Placement des objets dans la scène

Pour chaque type de relation, la position est calculée géométriquement en utilisant les dimensions des deux objets impliqués. Les distances entre ces deux objets sont soit données dans le fichier JSON, soit randomisées en tirant aléatoirement x et y dans la surface autorisée.

La fonction `apply_relation` convertit les relations textuelles données, en modifiant la position de chaque objet.

- Pour les relations verticales (*on* et *under*) la coordonnée z de l'objet du dessus est changée de sorte que le point le plus bas de cet objet soit situé au plus haut point de l'objet du dessous.
- Pour les relations horizontales (*against*, *right_of*, *left_of*, *in_front_of*, *behind*), les dimensions z sont changées de façon à ce que les objets soient à la même hauteur, et les dimensions x et y sont changées pour que la relation soit appliquée.

Pour décider dans quel ordre ces positions vont être appliquées, nous utilisons d'abord le graphe de scène, qui va nous permettre de déterminer l'objet le plus bas dans la scène, qui sera placé au centre de la scène. Les autres objets seront placés en relation. Afin d'avoir un plan plus stable, les dimensions horizontales (à droite, devant...) sont traitées avant les relations verticales, dans la mesure du possible (l'ordre du graphe de scène reste prioritaire).

La fonction `parent_bound` permet d'éviter que les objets soient placés en dehors des limites de leur parent. Dans cet algorithme, on calcule les limites de l'objet parent, sur les axes x et y , et on contraint la position de l'objet enfant pour qu'il reste dans ces limites, avec une marge de sécurité. On s'assure donc que malgré les autres relations, l'objet reste sur la surface sur laquelle il est censé rester.

4.4 Boucle de validation

Notre solution initiale était composée d'un ensemble d'appels VLM pour constituer une "chaîne de pensée", en se concentrant sur un problème à la fois : éviter les collisions, vérifier les orientations, et valider la scène sémantiquement. Cependant, après avoir exploré les possibilités des VLM pour la validation spatiale, nous avons constaté l'efficacité supérieure de l'usage de Genesis pour éviter les collisions en

utilisant les propriétés de l’environnement de simulation. Cette approche prend également beaucoup moins de temps, car un appel VLM sur machine locale demande un temps conséquent.

Notre boucle est donc maintenant définie par les étapes suivantes :

1. **Validation physique (Genesis) :** Les positions sont traitées par Genesis pour assurer la stabilité de la scène. Les collisions et flottaisons sont corrigées d’abord par l’analyse des bounding boxes (*AABB*), puis par la simulation. Un collage de vues de cette scène est généré pour le VLM.
2. **Validation sémantique (VLM) :** Le modèle compare l’état actuel de la scène générée avec le prompt ou la photo de référence. Il émet des instructions de haut niveau (*on* comme précédemment par exemple), qui sont ensuite converties en nouvelles coordonnées pour l’itération suivante.
3. **Convergence :** La boucle opère jusqu’à ce que le VLM valide la scène ou que le nombre maximum d’itérations soit atteint.

4.4.1 Validation physique

La validation physique est effectuée dans `jsonToSim.py`. Celle-ci résout les collisions et flottaisons d’objets.

Pour régler les chevauchements, l’algorithme applique un seuil latéral : si des objets se chevauchent en dessous du seuil, leur relation peut devenir une relation verticale : les objets sont empilés. Dans le cas d’un plus grand chevauchement, les objets sont écartés l’un de l’autre.

Ensuite, la simulation sur Genesis permet de détecter les objets instables. En effet, le comportement de Genesis induit qu’un objet enfoncé dans le sol va être expulsé vers le haut. Un objet généré trop haut et sans support va tomber. En simulant 150 pas de temps ($dt = 0.01$), on peut mesurer la vitesse verticale et le déplacement final de chaque objet, en utilisant la fonction Genesis `get_pos`. Les coordonnées de ces objets instables sont ensuite ajustées de façon à régler cette instabilité.

4.4.2 Sélection de l’angle de caméra (mode image)

En mode image, la comparaison VLM doit se faire avec un point de vue cohérent avec la photo de référence. La fonction `select_best_camera_angle` construit la scène initiale et effectue le rendu de 8 angles prédéfinis (`front`, `front_left`, `front_right`, `back`, `back_left`, `back_right`, `top_front`, `top`). Les 8 rendus et la photo de référence sont envoyés à `gpt-4o`, qui retourne le nom de l’angle le plus proche du point de vue de la référence. Cet angle est réutilisé pour tous les rendus suivants de la boucle. En cas de réponse invalide, l’angle `front` est utilisé par défaut.

4.4.3 Validation sémantique

La scène corrigée par la validation physique est transmise au VLM pour validation sémantique. La subtilité de cette étape repose sur le prompt système et sur la gestion des hallucinations. L’hallucination la plus fréquente étant celle des valeurs numériques, notre code interdit explicitement au VLM de manipuler des coordonnées : il ne peut renvoyer que des relations prédéfinies (`on`, `inside`, `right_of`, `left_of`, `in_front_of`, `behind`). Ces relations sont ensuite traduites en coordonnées physiques par `apply_semantic_corrections`, en s’appuyant sur les boîtes englobantes extraites précédemment et sur le graphe de scène pour confiner chaque objet aux limites de son parent (avec une marge `SURFACE_MARGIN = 0,04` m). Les relations latérales utilisent une distance fixe `DEFAULT_LATERAL_DISTANCE = 0,2` m.

Pour les relations `on` et `inside`, le modèle peut optionnellement fournir des ratios $rel_x, rel_y \in [0, 1]$ pour placer le sujet à une position relative précise sur la surface du support (par exemple coin supérieur droit). La position cible est alors calculée puis clampée :

$$target_x = ref_x - \frac{parent_w}{2} + rel_x \cdot parent_w$$

$$pos = \text{clamp}(target, ref_{center} - margin, ref_{center} + margin)$$

avec $margin = \max(0, (parent_{dim} - subject_{dim})/2 - SURFACE_MARGIN)$. Lorsque rel_x, rel_y ne sont pas fournis, les coordonnées ($prev_x, prev_y$) sont restaurées pour éviter une randomisation indésirable : seul z est corrigé.

Plusieurs garde-fous renforcent la robustesse face aux hallucinations. Le format de sortie est forcé via `response_format={"type": "json_object"}`, puis nettoyé (suppression des backticks, extraction du bloc `{...}`, normalisation `True/False` → `true/false`). Les corrections référençant un `subject` ou un `object` inconnu, typiquement une hallucination du VLM sont ignorées. Enfin, une liste de corrections vide force `valid` à `True`, ce qui constitue la condition de convergence effective de la boucle.



FIGURE 9 – Exemple : l’image initiale, les itérations 0, 3, 6 de la boucle de validation sur une scène, et le rendu final.

5 Résultats

Cette section présente l’évaluation expérimentale du système. Nous commençons par les métriques et le protocole (5.1), puis nous évaluons le pipeline texte sur les quatre versions V1, V1.1, V1.1.1 et V2 (5.2). Nous passons ensuite au pipeline image avec la comparaison V3 vs V3.1 sur les images du Franka (5.3), et nous évaluons la boucle de validation appliquée à V2 (5.4). Nous utilisons ensuite un questionnaire pour évaluer nos résultats sur les V2. Un tableau de synthèse comparative clôt la section (5.6).

5.1 Métriques et protocole expérimental

Données de test. L’évaluation repose sur trois jeux de données annotés manuellement. Le premier est un ensemble de 30 prompts en langage naturel, fourni dans `tests/data/prompts_ground_truth.json` par le Docteur Garrabé, où chaque prompt est associé à une liste d’objets attendus, à un ensemble de triplets de relations (`type`, `sujet`, `objet`) et à un ensemble de paires d’orientations (`id`, `turn`). Le second jeu est une banque de 7 images RGB acquises avec le robot Franka Emika Panda de l’ISIR, accompagnées des coordonnées 6D (lib Sam6D) ground truth fournies directement par ce dernier. Enfin, nous nous appuyons sur une étude de validation du système (formulaire) que nous avons réalisée sur 35 personnes.

Métriques. Les métriques varient selon la dimension que nous cherchons à évaluer. Le F1 reconnaissance compare l’ensemble des objets prédits à l’ensemble attendu. Le F1 relations fait la même chose sur les triplets (`type`, `sujet`, `objet`). L’exact match orientations mesure l’accord sur les paires (`identifiant`, `turn`). La latence est rapportée en moyenne, min et max sur 5 exécutions par prompt. Pour le pipeline image, nous rapportons l’erreur euclidienne de position $e_i = \|\hat{\mathbf{p}}_i - \mathbf{p}_i^{\text{GT}}\|_2$ en MAE, médiane et RMSE, ainsi qu’une décomposition par axe pour voir d’où vient l’erreur. Enfin, pour la boucle de validation, nous mesurons le taux de collisions avant et après la boucle, le taux de convergence (proportion de scènes qui atteignent `valid=True` en moins de 10 itérations), et les notes données par les répondants au formulaire sur la fiabilité du prompt et la cohérence physique.

Conditions matérielles et reproductibilité. Genesis tourne en backend CPU sur MacBook (Apple Silicon, Darwin 25.4.0, 8G RAM). Le fine-tuning de Phi-3 a été réalisé sur un GPU Tesla T4 via Google Colab, et le modèle affiné est servi en local via Ollama au format GGUF. Les appels GPT-4o sont faits avec `temperature=0` pour avoir des sorties reproductibles à snapshot constant. La graine 42 est fixée pour l’entraînement. Il reste un `random.uniform` non seedé dans `apply_relation` qui introduit une variabilité résiduelle dans les placements latéraux.

Limites. La cadre temporel de ce projet ne nous a permis de récolter qu’un nombre limité de données de tests (les ground truths doivent être annotées manuellement pour assurer la fiabilité).

5.2 Évaluation du pipeline texte

Cette sous-section regroupe l’évaluation des quatre versions V1, V1.1, V1.1.1 et V2 sur les 30 prompts de référence. Pour V1, V1.1 et V1.1.1, nous rapportons les résultats à la fois avec GPT-4o et avec Llama 3.1 pour quantifier l’effet du LLM sous-jacent. V2 utilise le modèle Phi-3 affiné en local (`phi3-scene`), et n’est donc présent qu’en une seule courbe. Chaque prompt est exécuté cinq fois et les métriques rapportées sont des moyennes sur ces cinq exécutions. Nous présentons successivement la reconnaissance d’objets (5.2.1), le placement et les orientations (5.2.2), puis la latence (5.2.3).

5.2.1 Reconnaissance d'objets

La reconnaissance est la première étape du pipeline et conditionne tout ce qui suit : un objet manqué à ce stade ne peut être ni placé ni orienté, et un objet halluciné se propage dans la suite de la chaîne. La figure 10 combine deux niveaux de lecture : les sous-figures (a) et (b) donnent le F1 prompt par prompt, ce qui permet de localiser les cas pathologiques, tandis que les sous-figures (c) et (d) agrègent le précision, le rappel et le F1 moyens.

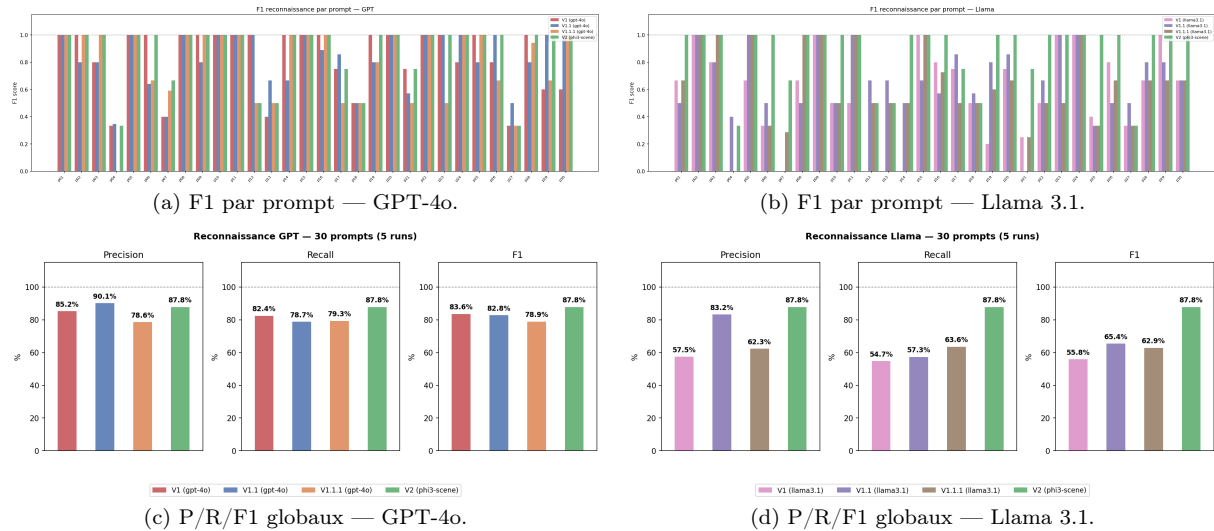


FIGURE 10 – Reconnaissance d'objets sur 30 prompts (5 runs/prompt). F1 par prompt (haut) et métriques agrégées (bas).

Avec GPT-4o, les quatre versions se tiennent dans une fourchette serrée, entre 78,9 % et 87,8 % de F1 global. V1 atteint 83,6 % en s'appuyant sur la connaissance directe du catalogue par le modèle, ce qui produit une précision élevée (85,2 %) au prix d'un rappel un peu plus faible (82,4 %) lié aux hallucinations d'identifiants. Ce résultat serait considérablement dégradé si la taille du catalogue dépassait le contexte du LLM(mauvais passage à l'échelle). V1.1 monte légèrement en précision (90,1 %) mais perd en rappel (78,7 %) à cause de la séquentialité de l'extraction puisque lorsque la validation sémantique du second appel rejette un mapping, l'objet correspondant disparaît pour le reste du pipeline. V1.1.1 baisse à 78,9 % de F1 en moyenne, parce que la résolution par embedding, plus permissive, fait remonter le rappel mais introduit aussi quelques confusions sur les objets dont les labels sont sémantiquement proches dans le catalogue(exp : `boite_plastique`, `boite_carton`..). V2 atteint enfin 87,8 % de F1, avec une précision et un rappel équilibrés. Le contraste est plus marqué sous Llama 3.1. La même architecture V1 chute à 55,8 % de F1, et V1.1 à 65,4. Le rappel s'effondre autour de 55 % sur V1 et V1.1, ce qui signifie qu'un objet sur deux du prompt n'est tout simplement pas reproduit dans la sortie. La résolution par embedding de V1.1.1 améliore le rappel (63,6 %) mais ne suffit pas à compenser, et le F1 reste autour de 63 %.

L'examen prompt par prompt fait ressortir quelques cas où toutes les versions chutent simultanément, notamment p04, p07 et p27 (`tests/data/prompts_ground_truth.json`) sous GPT-4o, et de nombreux prompts sous Llama dans la moitié inférieure du graphe. Ces cas concernent typiquement des objets pour lesquels le catalogue contient un identifiant proche mais non identique au label naturel attendu (par exemple `cardboard_box` face à un prompt mentionnant *une boîte carton ouverte*), ce qui pénalise les versions sans matching par embedding. V1.1.1 et V2, qui reposent toutes deux sur la similarité sémantique multilingue, sont les plus robustes à ce type de variation lexicale.

5.2.2 Placement : relations et orientations

Le placement est la dimension où les versions se différencient le plus, et c'est aussi la plus difficile à évaluer équitablement, parce que V1 et les versions suivantes ne produisent pas le même type de sortie. La version V1 produit directement des positions et orientations tridimensionnelles pour chaque objet, sans extraire explicitement de relations spatiales. Pour la comparer aux autres versions sur la métrique rel-F1, nous dérivons les relations implicites à partir des positions : pour chaque paire d'objets, nous comparons leurs coordonnées et leurs dimensions pour inférer si l'un est posé sur l'autre (axe z), à droite ou à gauche (axe y), ou devant ou derrière (axe x), en utilisant les dimensions réelles comme seuil de séparation. Ces relations dérivées sont ensuite évaluées avec le même `compute_relation_f1` que les autres versions, ce qui

permet une comparaison directe sur le même axe. Nous conservons donc en parallèle, à titre d'indicateur complémentaire, les métriques propres à V1 mesurées sur les sorties brutes : validité des quaternions, plausibilité physique de la coordonnée z , et ordre vertical correct. Sous GPT-4o, ces trois indicateurs atteignent respectivement 95,3 %, 92,0 % et 70,3 %, contre 90,0 %, 60,0 % et 61,7 % sous Llama 3.1, ce qui confirme que V1 produit certes des sorties syntaxiquement valides, mais que la cohérence spatiale fine se dégrade fortement avec un modèle moins puissant. À noter que nous n'évaluons pas V1.1.1 sur les relations et orientations puisque cette version n'apporte pas de modification à la précédente sur ces deux dimensions.

La figure 11 suit la même mise en page que la précédente. Les sous-figures (a) et (b) donnent le rel-F1 par prompt, et les sous-figures (c) et (d) agrègent en F1 relations et F1 orientations.

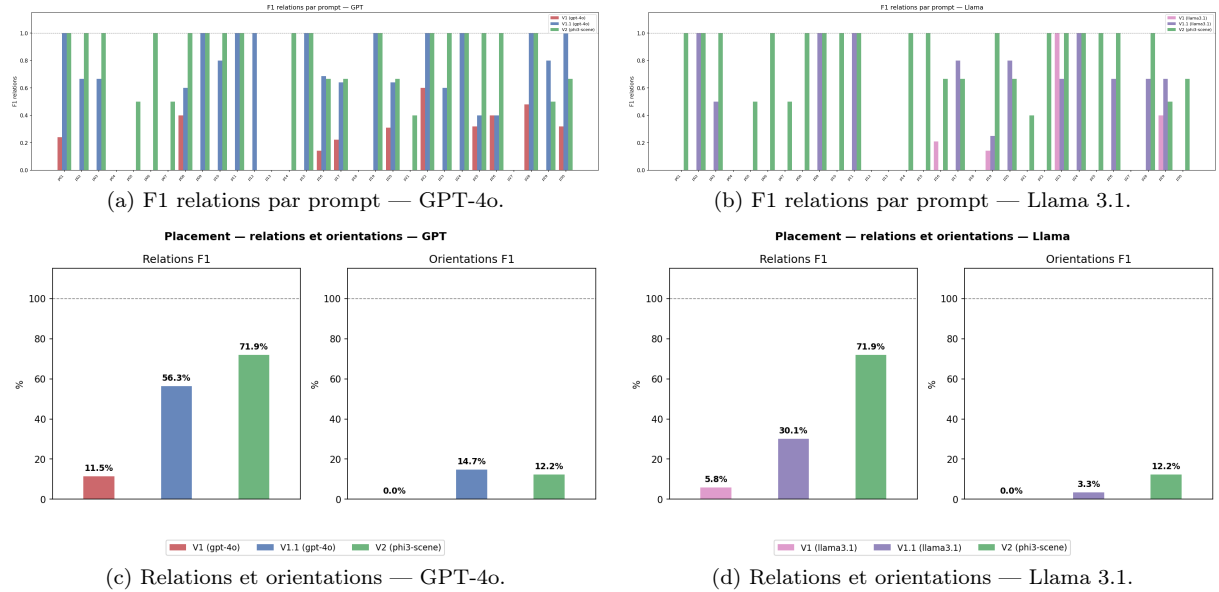


FIGURE 11 – Placement sur 30 prompts. Pour V1, les relations sont dérivées des positions 3D (voir texte). V1.1.1 absent (matching catalogue uniquement).

L'écart entre versions est cette fois bien plus net que pour la reconnaissance. Sous GPT-4o, V1 ne dépasse pas 11,5 % de rel-F1, ce qui était attendu compte tenu du caractère peu précis de la dérivation à partir des positions, V1.1 monte à 56,3 % grâce à l'extraction explicite des triplets, et V2 atteint 71,9 %. Sous Llama 3.1, la même hiérarchie se retrouve à des valeurs plus faibles. Pour les orientations, les scores restent bas sur toutes les versions (0 % pour V1, 14,7 % pour V1.1 GPT, 3,3 % pour V1.1 Llama, 12,2 % pour V2), ce qui reflète surtout la rareté des orientations dans le benchmark, c'est une section que nous n'avons pas beaucoup traité pour le moment. V2 reste néanmoins l'option la plus fiable sur les prompts concernés.

L'examen prompt par prompt fait ressortir cinq cas (p04, p05, p13, p14, p18) où toutes les versions sont à zéro, et qui correspondent aux deux limites structurelles connues du système. La première concerne les relations rares hors de notre vocabulaire supporté : p04 (**under**) ou p05 (**against**) sont systématiquement interprétée par **on** (inversé) ou **next_to**, notre affinage n'a pas encore été enrichi sur ces cas. La seconde concerne l'extraction de distances numériques explicites (p13, p14, p18), ignorées par les LLMs généralistes et insuffisamment représentées dans notre dataset synthétique pour que **phi3-scene** les restitue de manière fiable. Ces cinq cas constituent donc les limites de capacité actuel du système.

5.2.3 Latence

La latence est mesurée comme le temps total du pipeline, depuis la réception du prompt jusqu'à la production de la scène JSON (sans construction genesis). Les mesures sont effectuées sur CPU local. La figure 12 reporte le temps total par prompt et le tableau 4 agrège en moyenne, minimum et maximum.

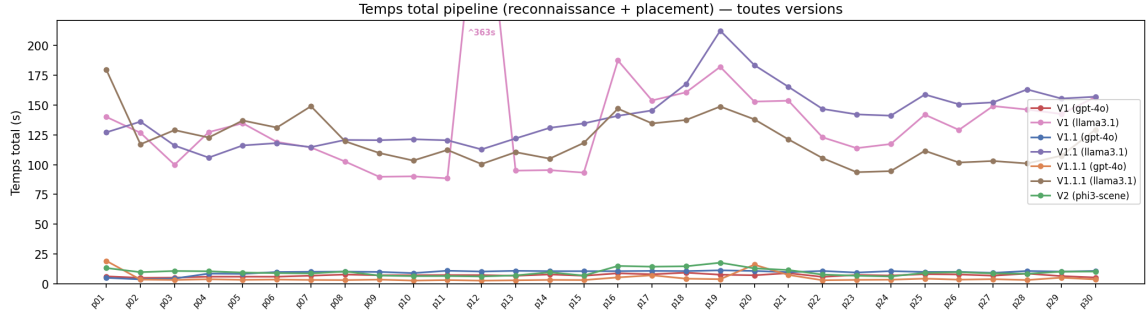


FIGURE 12 – Latence totale du pipeline (reconnaissance + placement) par prompt. V1.1.1 ne couvrant que la reconnaissance, son temps n’est pas directement comparable aux autres pipelines complets. La courbe V1 sous Llama présente un pic isolé à 363 s sur un prompt, signalé en annotation.

TABLE 4 – Latence totale du pipeline sur 30 prompts (en secondes).

Pipeline	Moy. Rec	Moy. Rel	Moy.	Min	Max
V1 GPT-4o	3,0	4,0	7,1	5,0	9,3
V1 Llama	42,7	93,7	136,4	88,5	363,5
V1.1 GPT-4o	3,3	6,3	9,6	3,8	11,3
V1.1 Llama	51,0	89,0	140,0	105,9	212,4
V1.1.1 GPT-4o	1,1	3,6	4,7	2,7	19,3
V1.1.1 Llama	11,5	109,1	120,7	93,7	179,6
V2 phi3-scene	—	—	9,8	6,2	17,7

Deux régimes de latence : le premier, autour des 5-10s, est celui des versions GPT-4o par appels API : V1 à 7,1 s, V1.1 à 9,6 s, le deuxième, autour de 130 à 140 s, est celui des mêmes pipelines exécutés avec Llama 3.1 en local. Ce coût s’explique par la taille du modèle (8 milliards de paramètres) et par le backend CPU. V2 s’établit en moyenne à 9,8 s pour le pipeline complet, soit l’ordre de grandeur des versions GPT-4o, malgré une exécution entièrement locale. En regroupant les multiples appels LLM des versions précédentes en une seule requête à **phi3-scene** et en s’appuyant sur un modèle dimensionné pour la tâche (3,82 milliards de paramètres en quantification 4-bit), V2 supprime le facteur dix à quinze de surcoût que payaient les versions Llama, tout en se passant complètement de l’API.

Les trois axes d’évaluation convergent vers une même conclusion. Le pipeline local affiné V2 rivalise en temps et performance avec un modèle cloud volumineux (voire le surpasse sur la dimension qui nous intéresse le plus, à savoir la cohérence des relations spatiales extraites), tout en restant local. Cette convergence valide a posteriori le pari du fine-tuning ciblé.

5.3 Évaluation du pipeline image

L’évaluation des deux versions du pipeline "image" est faite sur 7 images RGB acquises avec le robot Franka Emika Panda de l’ISIR (dont une de calibration). Pour chaque image, le robot fournit les coordonnées en six dimensions (trois positions et trois orientations) ground truth des objets présents dans la scène, ce qui nous permet d’évaluer V3 et V3.1 de façon métrique et absolue, et pas seulement qualitative comme c’est souvent le cas pour les pipelines image en l’absence de profondeur explicite.

5.3.1 Protocole

Pour chaque image d’entrée, le pipeline (V3 ou V3.1) produit une scène JSON contenant les positions 3D estimées des objets reconnus. Ces positions sont alignées au mieux sur le repère du robot Franka via une transformation de calibration à partir d’une image de référence ; les résultats sont donc à nuancer. La métrique principale est l’erreur euclidienne de position e_i , définie dans la section 5.1. Nous rapportons MAE, médiane et RMSE sur l’ensemble des objets correctement reconnus, ainsi que la décomposition par axe (e_x , e_y , e_z) afin de voir si l’erreur provient principalement de la profondeur, ce qui est attendu. Nous ne testons pas ici les orientations, car il s’agit d’une section que nous n’avons pas encore traitée dans le pipeline d’images.

5.3.2 Résultats V3 vs V3.1

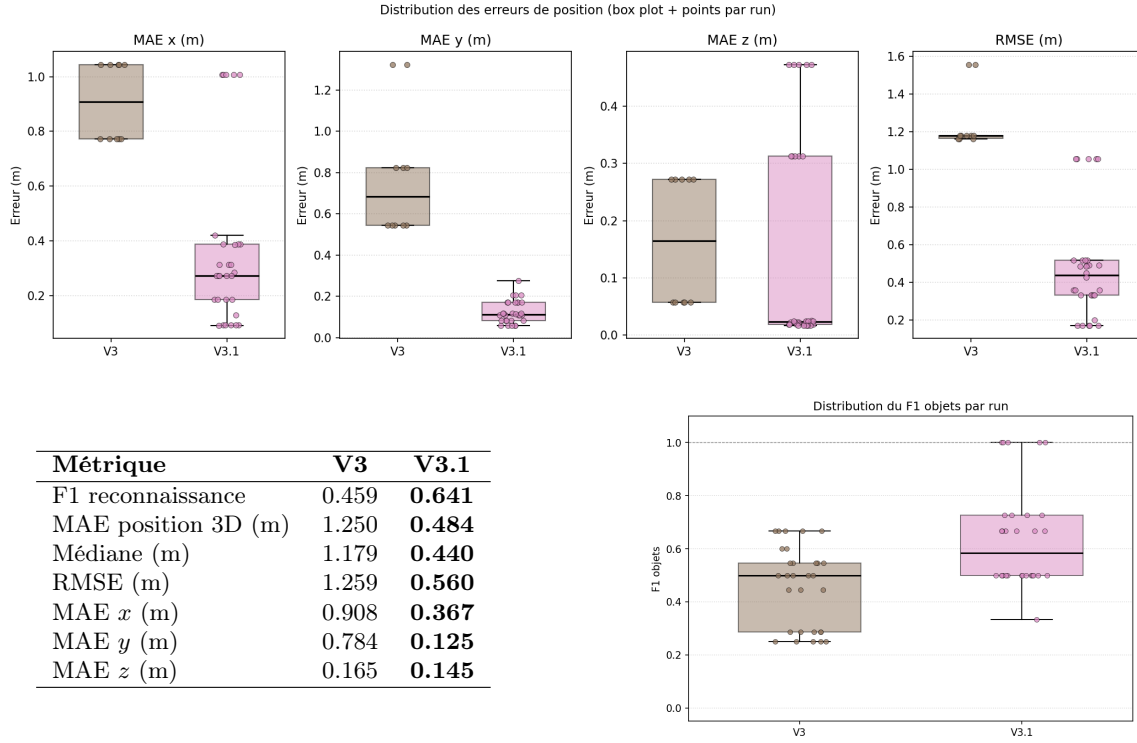


FIGURE 13 – Distribution des erreurs de position par axe (haut) et du F1 de reconnaissance d’objets (bas, droite), pour V3 et V3.1 sur les images du Franka. Le tableau donne l’évaluation métrique de V3 et V3.1 sur 7 images du Franka. V3 : 2 images, 10 runs ; V3.1 : 6 images, 30 runs.

Erreurs métriques par axe.

- **MAE x** : V3 forme une boîte large centrée vers 0,9 m avec une dispersion 0,7–1,3 m. V3.1 a une médiane à $\approx 0,30$ m mais une queue longue vers 1,0 m (outlier). La majorité des résultats de V3.1 sont en dessous du pire cas de V3, soit un gain net.
- **MAE y (latéral)** : c’est ici que le gain est le plus conséquent. V3 produit des erreurs entre 0,5 et 1,3 m, V3.1 reste serrée sous 0,25 m, soit un ordre de grandeur de moins. Ce résultat n’est pas surprenant puisque la formule $y_{\text{world}} = (u - c_x)/f_x \cdot D$ utilise directement la profondeur D fournie par la *depth map*, et c’est précisément le calcul que V3 devait deviner à l’œil.
- **MAE z (hauteur)** : les deux versions sont équivalentes, autour de 0,15 m. C’est ce qui est attendu, z est calculé en aval par Genesis à partir des AABB des objets, sans faire intervenir ni le VLM ni la *depth map*. V3.1 ne pouvait donc rien améliorer sur cet axe.
- **RMSE** : la dispersion globale de V3.1 est environ 2,5 fois plus serrée que celle de V3. V3.1 elle réduit donc drastiquement la variance sur x et y en plus d’améliorer la moyenne.

F1 reconnaissance d’objets.

- **V3** : médiane $\approx 0,50$, étalée entre 0,25 et 0,67. Aucun run parfait.
- **V3.1** : médiane $\approx 0,55$, mais surtout un cluster à 1,0 (*img02* parfaite sur 5 runs, dans *tests/images_lab0*) et un cluster bas vers 0,4–0,5. V3.1 réussit complètement ou échoue partiellement)

Synthèse. V3.1 améliore sur tous les axes où ses primitives ajoutées agissent (reconnaissance par embeddings, contours *GroundingDINO*, back-projection métrique via *depth map* ou projection orthographique), et son seul axe “invariant” (z) est le seul où elle n’apporte rien algorithmiquement. Le surcoût en temps (3,3 s $\rightarrow$ 22 s par image (avec GPT-4o)), du fait de l’empilement des quatre modèles, est largement compensé.

Étude de cas. La figure 14 illustre une scène typique du benchmark : à gauche l'image RGB d'entrée acquise par la caméra du robot Franka, à droite la scène générée par V3.1 dans Genesis. Trois objets sur quatre sont reconnus. Les différences notables sont encadrées : en vert et bleu, le tournevis et les ciseaux sont relativement bien positionnés mais leurs orientations sont par défaut ; en rouge, le VLM n'a pas pu reconnaître le mug du fait de sa similarité de couleur avec la boîte de conserve.

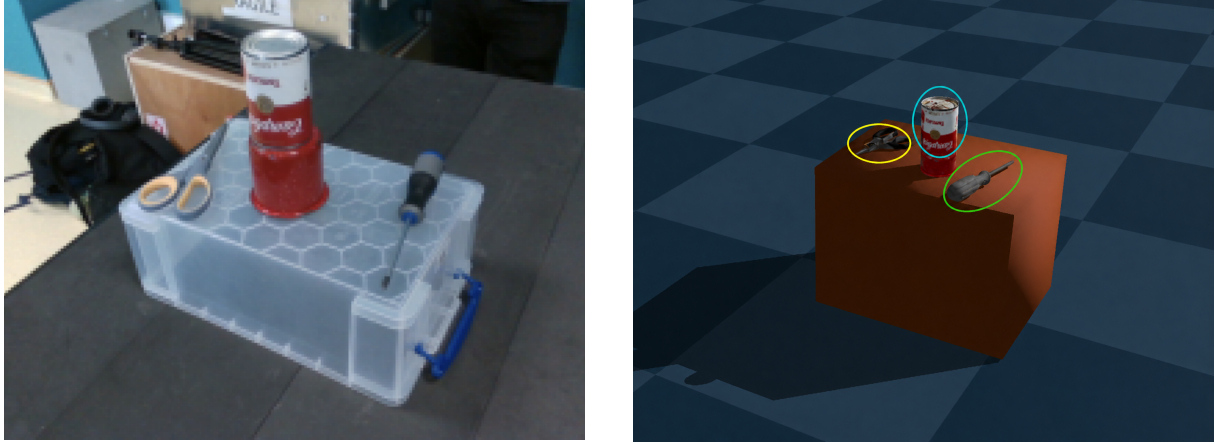


FIGURE 14 – Exemple représentatif du benchmark Franka : image RGB d'entrée (gauche), scène Genesis reconstruite par le pipeline image (droite).

5.4 Évaluation de la boucle de validation physique et sémantique

La boucle de validation n'est évaluée que sur V2, qui est la version par défaut du système. L'évaluation repose sur les 30 prompts du benchmark texte et combine deux protocoles : (i) la mesure du taux de collisions avant et après boucle, qui quantifie l'apport de la validation physique par Genesis ; (ii) la mesure du taux de convergence de la boucle.

5.4.1 Validation physique et effet sur les relations

Nous comparons l'état initial de chaque scène, en sortie de la construction géométrique, à son état final après convergence ou plafonnement à dix itérations. Deux indicateurs sont étudiés : le nombre de paires d'objets en collision détectées par Genesis, et le pourcentage de relations *ground truth* du prompt effectivement présentes dans la scène.

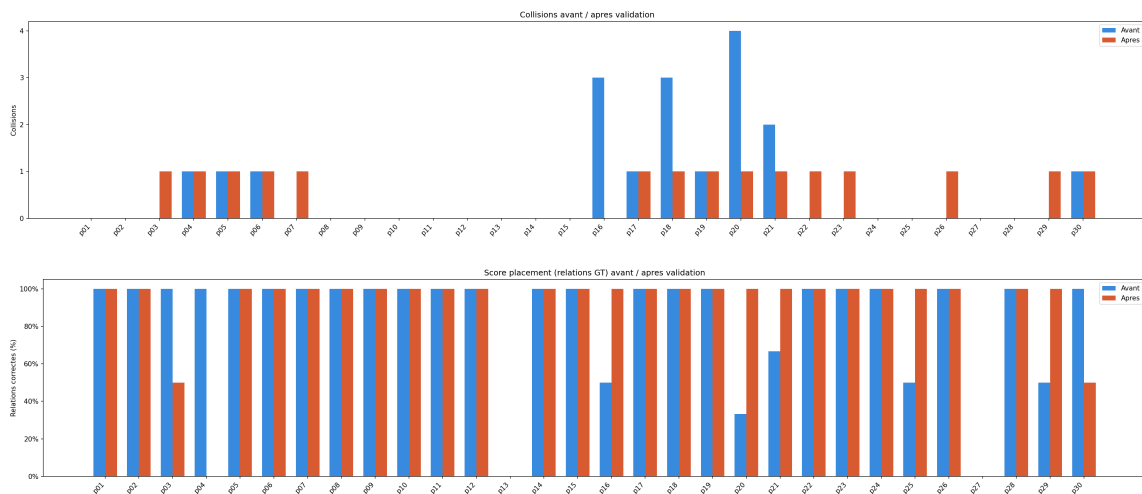


FIGURE 15 – Effets de la boucle de validation sur l'état final des scènes (30 prompts, V2) : collisions détectées (haut) et relations *ground truth* respectées (bas), avant et après boucle.

La boucle remplit son rôle sur les cas les plus dégradés en sortie de construction. Les quatre scènes initialement les plus en collision (p16 et p18 avec 3, p20 avec 4, p21 avec 2) sont ramenées à au plus une

paire en chevauchement, et ces mêmes prompts retrouvent une conformité relationnelle complète (p20 passe de 33 % à 100 %, p21 de 67 % à 100 %, p16 de 50 % à 100 %). Sur ces cas, gain physique et gain sémantique se renforcent : la correction qui résout la relation manquante évacue la collision qu'elle causait.

En contrepartie, la boucle introduit une collision sur six scènes initialement propres (p03, p07, p22, p23, p26, p29) et fait régresser deux scènes en placement (p03 et p30, de 100 % à 50 %). La proportion de scènes sans collision recule de 66,7 % à 50,0 %, et le placement moyen ne progresse que modestement, de 91,1 % à 92,9 %.

Parmi deux éléments problématiques que nous avons identifiés : sur p22 et p23, le défaut perçu est orientationnel (un objet n'a pas été correctement retourné par exemple) mais le module VLM de la boucle ne traduit que des relations de position pour le moment. Le VLM émet alors des *on* parasites qui re-placent les objets en vain. Sur p03 et p29 (clé USB près d'un laptop), la distance latérale par défaut (`DEFAULT_LATERAL_DISTANCE` = 0,2 m), soit la distance qu'applique le VLM lorsqu'il corrige une relation spatiale non-verticale, ignore la taille effective des objets : l'écart dépasse le bord du support, et le *clamp* ramène la clé contre le laptop. Ce sont des éléments que nous corrigerons dans la suite de ce travail.

5.4.2 Convergence de la boucle

Le deuxième axe d'analyse quantifie le nombre nécessaires pour atteindre la validation, et quelle proportion des scènes y parvient avant le plafond (figure 16).

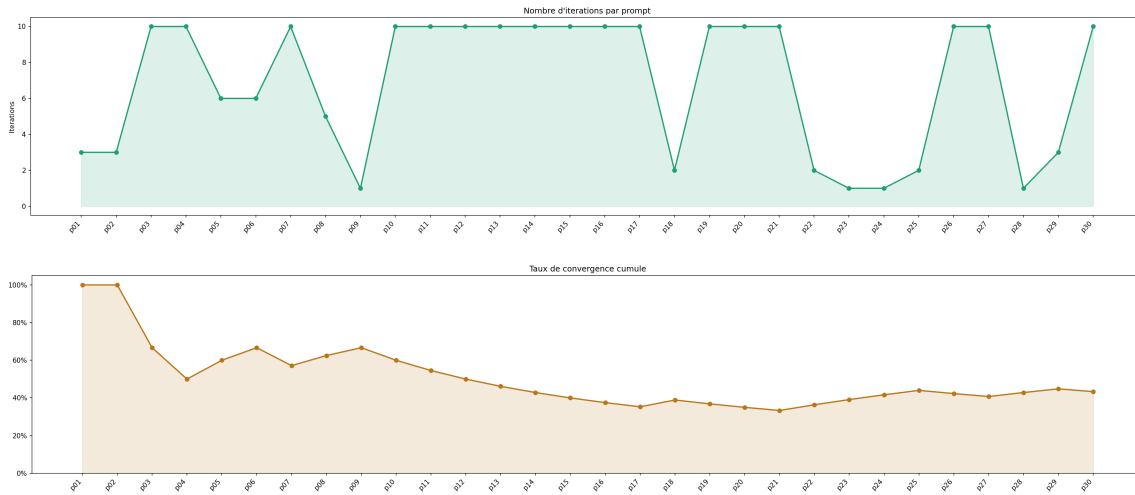


FIGURE 16 – Dynamique de convergence de la boucle de validation sur 30 prompts : itérations effectuées par prompt (haut, plafond 10) et taux de convergence cumulé (bas).

Sur les 13 scènes qui convergent, douze le font en trois itérations ou moins, et la zone intermédiaire, entre quatre et neuf itérations, est presque vide. Soit le VLM cesse rapidement de proposer des corrections et la scène est validée, soit il en propose indéfiniment et la boucle plafonne à dix. Nous avons identifié deux raisons à cela : soit la boucle ne s'améliore pas progressivement pour les cas trop difficiles, soit le VLM ne reconnaît pas un ou plusieurs objets de la scène et considère le placement comme faux. Comme nous avons choisi de ne pas donner à la boucle la liberté de modifier les objets sélectionnés (plusieurs itérations entraînent trop d'hallucinations), elle boucle sans fin, voire applique de mauvaises corrections. Ces limites sont discutées dans la section 6.

5.4.3 Synthèse

Le tableau 5 récapitule l'ensemble des indicateurs. La boucle fait ce pour quoi elle a été dimensionnée sur les cas pathologiques de placement initial, où gain physique et gain sémantique se conjuguent, et améliore légèrement le placement moyen (de 91,1 % à 92,9 %). Cependant, elle dégrade parfois des scènes déjà correctes, et consomme l'intégralité de ses dix itérations sur les scènes hors capacité, ce qui limite le taux de convergence à 43,3 %.

Métrique	Valeur
<i>Validation physique et placement</i>	
Collisions moyennes par scène (avant)	0,60
Collisions moyennes par scène (après)	0,50
Scènes sans collision (avant)	66,7 %
Scènes sans collision (après)	50,0 %
Score placement moyen (avant)	91,1 %
Score placement moyen (après)	92,9 %
<i>Convergence</i>	
Nombre moyen d'itérations	6,87
Médiane des itérations	10
Taux de convergence (< 10 itérations)	43,3 %

TABLE 5 – Récapitulatif des indicateurs de la boucle de validation sur 30 prompts (V2).

5.5 Validation sémantique du système par étude utilisateur

Pour évaluer la cohérence sémantique, 27 scènes finales (nous avons dû enlever 3 scènes pour manque d'un objet dans notre librairie) ont été présentées par le biais d'un questionnaire Google Forms à 35 répondants qui ont noté de 1 à 10 la *fidélité au prompt* et la *cohérence physique* de la scène. Le choix d'une étude utilisateur plutôt qu'une validation purement automatisée via VLM est motivé par notre volonté de conserver l'intégrité de nos tests, sachant que les VLM pourraient halluciner.

La figure 17 montre une comparaison des moyennes des deux notes (fidélité au prompt et cohérence physique) par scène. Nous pouvons y identifier les scènes bien notées et celles qui ont posé problème à une majorité de répondants. Cette figure permet de voir une différence entre les deux variables d'évaluation selon les prompts, et de constater que les notes de fidélité et cohérence sont souvent corrélées. Nous pouvons également voir une grande différence entre les notes de différents prompts.

Par exemple, *p23*, le prompt "*A laptop turned to the right on a washing machine*", obtient un mauvais score car la relation a été confondue par notre système. L'image montrait ainsi un ordinateur *en dessous* d'un lave-linge. La mauvaise évaluation des prompts 09 et 12 pour la fidélité au prompt est due à une erreur de choix des objets pour la scène. Les prompts, "*Une poubelle à 50 cm à gauche d'un réfrigérateur*" et "*A toaster is 70 cm to the right of an oven.*", affichent respectivement une perceuse à gauche d'une poubelle, et deux toasters côte à côte.

Cela montre que la performance de notre système dépend encore de la complexité de la tâche demandée. Le questionnaire nous permet toutefois de montrer une bonne performance sémantique et physique, avec des notes moyennes respectives de 8/10 et 8.17/10 comme montré dans le tableau récapitulatif 6.

Il est important de noter que, par contrainte de temps et de moyens, nos répondants ne faisaient pas forcément partie du public visé pour ce projet. Ainsi, il serait intéressant de réitérer cette expérience, avec des experts en robotique et en simulation.

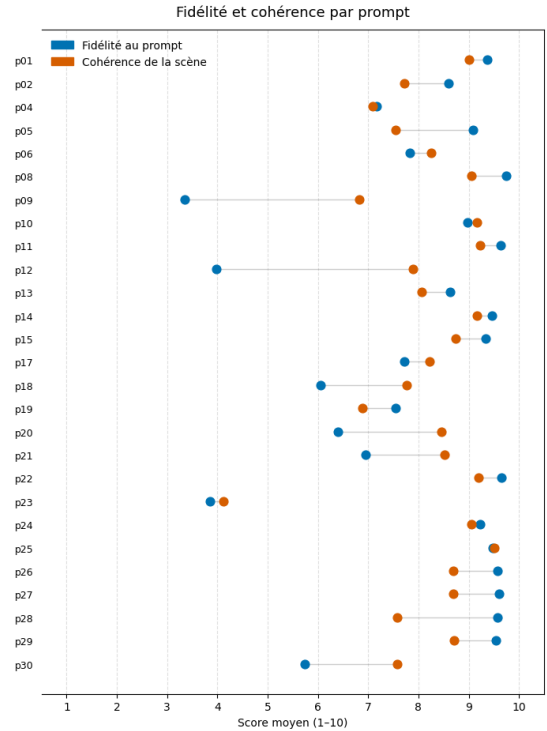


FIGURE 17 – Cleveland box plot montrant la moyenne des notes par scène.

Métrique	Valeur
Nombre de répondants	35
Nombre de scènes évaluées	27
Note moyenne de fiabilité du prompt (/10)	8.00
Note moyenne de cohérence physique (/10)	8.17
Médiane fiabilité du prompt	9
Médiane cohérence physique	9
Écart-type fiabilité du prompt	2.38
Écart-type cohérence physique	2.24

TABLE 6 – Résultats de l’étude utilisateur. Les 27 scènes produites par V2 + boucle ont été évaluées par 35 répondants.

5.6 Synthèse comparative

Le tableau 7 résume l’ensemble des résultats en croisant chaque version du pipeline avec les principaux axes d’évaluation. Il sert de référence pour la discussion dans la section 6.

Version	F1 obj.	rel f1	ori f1	Latence moy.	Coût rel.	Image
V1 (GPT-4o)	83.6%	11.5%	0.0%	7.1s	élevé	non
V1.1 (GPT-4o)	82.8%	56.3%	14.7%	9.6s	élevé	non
V1.1.1 (GPT-4o)	78.9%	56.3%	14.7%	1.1s	élevé	non
V2 (phi3-scene)	87.8%	71.9%	12.2%	9.8s	nul	non
V3	45.9%	—	—	3s	élevé	oui
V3.1	64.1%	—	—	22s	élevé	oui

TABLE 7 – Synthèse comparative des versions du pipeline. Le coût relatif est qualitatif : nul correspond à une inférence locale sans appel API (V2), élevé correspond aux versions qui appellent GPT-4o à chaque scène. Pour V3 et V3.1, nous n’évaluons pas le placement relatif au sens de **rel_f1**, le ground truth image étant exprimé en coordonnées métriques absolues et non en relations symboliques.

6 Discussion

Nous avons identifié plusieurs pistes d’amélioration pour ce travail.

1. **Gestion de l’échelle des objets** : notre logiciel ne permet pas de changer la taille d’un objet. Ajouter ce paramètre rendrait les scènes plus personnalisables. Par exemple, à partir d’un seul modèle de boîte, on pourrait générer des tables de différentes tailles.
2. **Rotations plus complexes** : pour l’instant, seules des rotations multiples de quart de tour sont possibles. Un système plus fin permettrait des orientations précises. Il faudrait pour cela limiter les hallucinations du LLM sur les valeurs numériques d’angles ou utilisé un module spécialisé.
3. **Relations plus complexes** : nous ne gérons que des relations simples (sur, dessous, à côté...). Des relations plus riches, comme suspendre un objet à un porte-manteau, exigeraient un travail conjoint sur orientations et relations. La gestion de relations multi-objets (disposition en cercle) ou la reconnaissance de symétries sont aussi des pistes intéressantes.
4. **Améliorer l’estimation de profondeur de V3.1** : la pipeline V3.1 utilise actuellement **Depth-Anything-V2-Metric-Indoor-Small**, un modèle léger. Passer à un modèle plus performant (version Large, ou un modèle métrique plus récent type **Metric3D**, **UniDepth**) améliorerait la précision du placement 3D, en particulier pour les relations de profondeur (**in_front_of**, **behind**).
5. **Élargir les angles de vue gérés par V3.1** : pour le moment, V3.1 ne gère que deux types de vue : **frontal** et **top_down**. Toute image prise sous un angle intermédiaire est ramenée à l’un de ces deux cas, ce qui dégrade la reconstruction. Il faudrait généraliser à des angles de caméra arbitraires, en estimant directement la pose complète (hauteur, tilt, roll, fov) puis en projetant les objets selon cette pose.
6. **Enrichir le finetuning de V2** : le modèle **phi3-scene** de V2 a été finetuné sur seulement un peu de 3000 exemples, sans suréchantillonner les cas difficiles. Les scènes complexes et les orientations sont donc sous-représentées. Augmenter le volume de données, équilibrer les classes de relations, et inclure plus d’exemples d’orientations devrait améliorer significativement les performances.

7. **Améliorer la boucle de validation** : la boucle actuelle corrige les scènes par itérations successives, mais reste limitée : critères d'arrêt rigides, peu de retours visuels exploités, et pas de mémoire entre itérations. Des pistes : intégrer une comparaison visuelle plus fine (depth + segmentation), exploiter les itérations passées comme contexte, et adapter dynamiquement le nombre d'itérations à la difficulté de la scène.

7 Conclusion

Ce projet pose les bases d'un outil de génération automatique de scènes pour la simulation robotique sous Genesis. En supprimant les deux obstacles principaux (le temps de construction manuelle et l'expertise technique requise), il propose une voie rapide entre une intention utilisateur (prompt textuel ou photo d'une scène réelle) et un environnement simulable.

Nous avons exploré plusieurs approches complémentaires : un pipeline LLM pur (V1), une variante hybride avec primitives géométriques (V1.1) pour éviter les hallucinations numériques, un modèle fine-tuné spécifique au domaine (V2), et un pipeline vision-langage prenant des images en entrée (V3, V3.1). À chaque version, le principe directeur est resté le même : déléguer au modèle ce qu'il sait faire (perception, reconnaissance, raisonnement qualitatif), et calculer de manière déterministe ce qu'il ne maîtrise pas (coordonnées métriques, dimensions, collisions). Une boucle de validation physique et sémantique corrige les incohérences résiduelles.

À terme, ce type d'outil pourrait permettre à des laboratoires de robotique de générer à grande échelle des environnements d'entraînement variés, sans la charge de modélisation manuelle qui freine aujourd'hui cette démarche. C'est un pas vers la démocratisation de l'apprentissage robotique simulé, en particulier pour les structures à moyens limités, et un levier potentiel d'accélération du transfert sim-to-real des politiques apprises.

Références

- [1] Zhou XIAN et al. *Genesis : A Generative and Universal Physics Engine for Robotics and Beyond*. <https://github.com/Genesis-Embodied-AI/Genesis>. 2024.
- [2] Matt DEITKE et al. “ProcTHOR : Large-Scale Embodied AI Using Procedural Generation”. In : *Advances in Neural Information Processing Systems (NeurIPS)*. T. 35. 2022.
- [3] Yue YANG et al. “Holodeck : Language Guided Generation of 3D Embodied AI Environments”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024.
- [4] Yufei WANG et al. “RoboGen : Towards Unleashing Infinite Data for Automated Robot Learning via Generative Simulation”. In : *arXiv preprint arXiv :2311.01455* (2023).
- [5] Boyuan CHEN et al. “SpatialVLM : Endowing Vision-Language Models with Spatial Reasoning Capabilities”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2024.
- [6] Josh TOBIN et al. “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World”. In : *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2017.
- [7] Fereshteh SADEGHI et Sergey LEVINE. “CAD2RL : Real Single-Image Flight without a Single Real Image”. In : *Robotics : Science and Systems (RSS)*. 2017.
- [8] Marah ABDIN et al. “Phi-3 Technical Report : A Highly Capable Language Model Locally on Your Phone”. In : *arXiv preprint arXiv :2404.14219* (2024).
- [9] Tim DETTMERS et al. “QLoRA : Efficient Finetuning of Quantized LLMs”. In : *arXiv preprint arXiv :2305.14314* (2023).
- [10] Shilong LIU et al. “Grounding DINO : Marrying DINO with Grounded Pre-Training for Open-Set Object Detection”. In : *arXiv preprint arXiv :2303.05499* (2023).
- [11] Berk CALLI et al. “The YCB Object and Model Set : Towards Common Benchmarks for Manipulation Research”. In : *IEEE International Conference on Advanced Robotics (ICAR)*. 2015.
- [12] Kaichun MO et al. “PartNet : A Large-scale Benchmark for Fine-grained and Hierarchical Part-level 3D Object Understanding”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2019.
- [13] Aaron GRATTAFIORI et al. “The Llama 3 Herd of Models”. In : *arXiv preprint arXiv :2407.21783* (2024).
- [14] Liang WANG et al. “Multilingual E5 Text Embeddings : A Technical Report”. In : *arXiv preprint arXiv :2402.05672* (2024).
- [15] Nils REIMERS et Iryna GUREVYCH. “Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks”. In : *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. 2019.
- [16] Edward J. HU et al. “LoRA : Low-Rank Adaptation of Large Language Models”. In : *arXiv preprint arXiv :2106.09685* (2021).
- [17] Daniel HAN, Michael HAN et UNSLOTH TEAM. *Unsloth : 2x faster fine-tuning, 70% less memory*. <https://github.com/unslothai/unsloth>. 2024.

1 Annexe 1 : Manuel d'utilisation

Cette partie détaille les consignes d'utilisation de notre logiciel depuis son interface. Celui-ci permet la génération d'environnements de simulation robotique depuis un prompt image ou texte.

1.1 Vue d'ensemble de l'interface

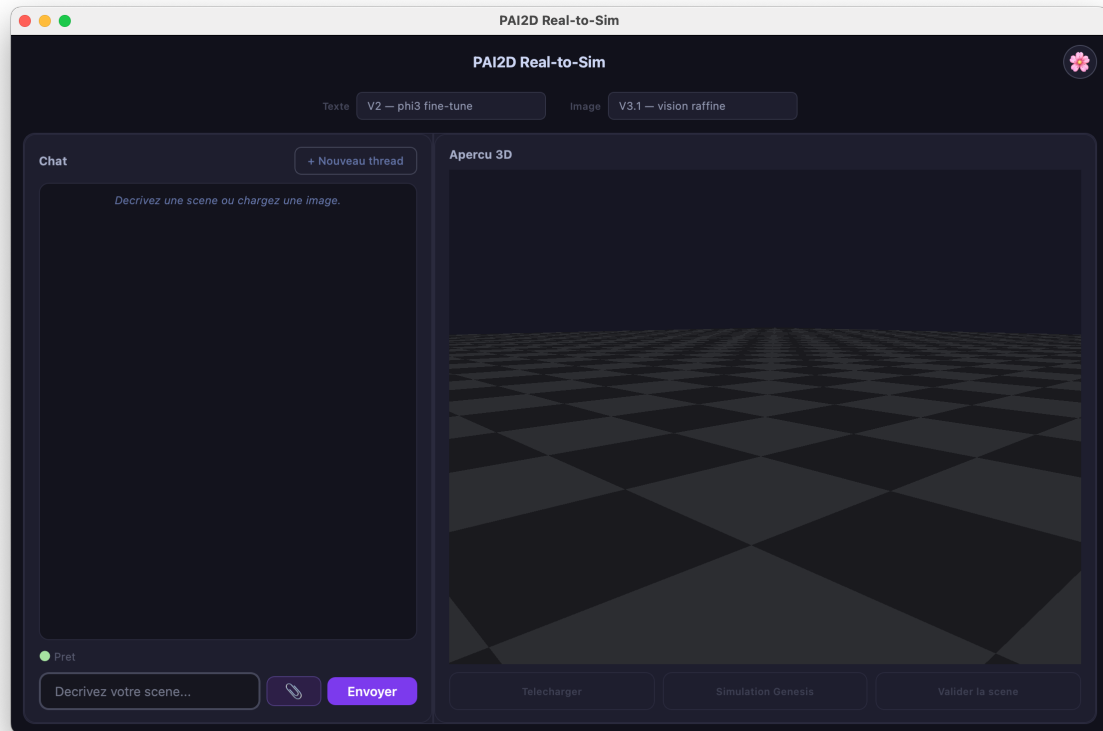


FIGURE 18 – Vue complète de l'interface.

L'interface, comme vu dans la figure 18, est divisée en quatre zones principales :

- **Barre de sélection des modèles** : permet de choisir parmi les modèles et pipelines disponibles.
- **Panneau de Chat** : permet la saisie des instructions sur la scène, ainsi que la suivie des étapes de la génération de scène effectuée par le programme.
- **Aperçu 3D** : permet une première visualisation rapide de la scène générée.
- **Barre d'actions** : téléchargement, simulation sur Genesis, et validation de la scène.

La figure 19 montre ces quatre zones différentes.

1.2 Description des éléments

1.2.1 Barre de sélection des modèles

Cette barre permet de choisir quel pipeline utiliser pour générer la scène. La sélection de la pipeline texte et de la pipeline image se fait séparément dans les panneaux respectifs. **Attention** : le choix de modèles doit être effectué avant l'envoi du premier message.

Quand utiliser le modèle texte : lorsque l'on veut décrire la scène demandée à l'écrit.

Quand utiliser le modèle image : lorsque l'on a une image de la scène désirée.

1.2.2 Panneau de chat

Il s'agit du point principal pour interagir avec le système, avec les fonctionnalités suivantes :

- **Nouveau thread** : permet de commencer une nouvelle conversation et de générer une autre scène. Ce bouton crée une nouvelle session de travail.

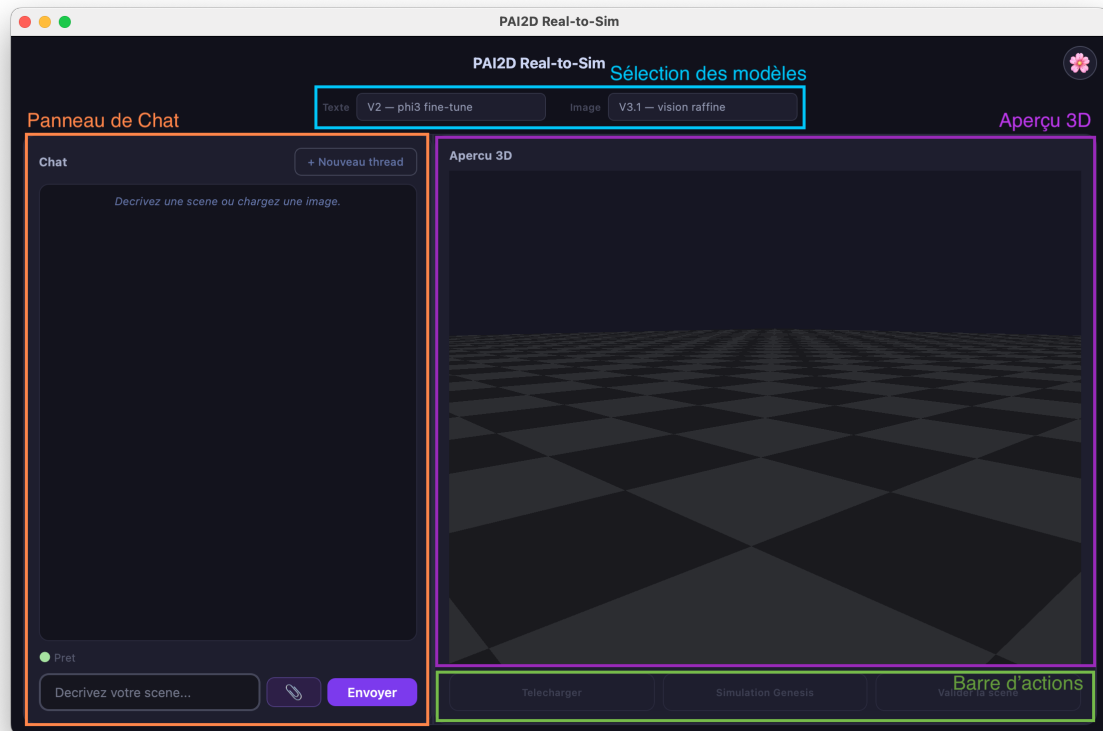


FIGURE 19 – Vue complète de l'interface.

- **Zone de saisie** : le champ "décrivez votre scène..." en bas du panneau permet à l'utilisateur d'écrire les instructions voulues. Une bonne pratique est de décrire les positions relatives des éléments entre eux, et d'éviter les instructions trop vagues.
- **Joindre un image** : l'icône *trombone* à gauche du bouton *Envoyer* permet de donner un image à répliquer pour créer la scène, plutôt qu'un prompt. **Bouton "Envoyer"** : permet l'envoi des instructions par l'utilisateur. **Indicateur d'état** : cette icône, verte ou orange, permet de signaler respectivement que le système est disponible pour l'envoi d'une instruction, ou qu'une génération est déjà en cours.

1.2.3 Aperçu 3D de la scène

Ce panneau, situé à droite, affiche rapidement la scène générée. Une mouvement *drag and drop* permet de faire pivoter la vue sur la scène.

1.2.4 Validation de la scène

Ce bouton, qui devient actif lorsqu'une scène est générée, permet d'effectuer une validation automatique de la cohérence physique et sémantique de la scène générée. Cette validation corrige les comportements physiques incorrects ainsi que des erreurs de placement des objets.

1.2.5 Simulation Genesis

Ce bouton est utilisé pour lancer la scène sur le moteur de simulation Genesis. Une nouvelle fenêtre est ouverte, contenant une session de simulation physique.

1.2.6 Téléchargement de la scène

Le bouton *Télécharger* permet l'export de la scène générée.

1.3 Workflow typique

Nous recommandons le workflow suivant pour créer et exporter une scène :

1. **Sélectionner le modèle** adapté à votre entrée (texte ou image) dans la barre du haut.
2. **Décrire la scène** dans le champ de saisie, ou joindre une image via le trombone.
3. **Envoyer** et patienter que l'indicateur repasse au vert.
4. **Inspecter l'aperçu 3D** et affiner la description si nécessaire.
5. **Valider la scène** via le bouton *Valider la scène*.
6. **Lancer la simulation Genesis** avec le bouton *Simulation Genesis*.
7. **Télécharger** le fichier de scène.

2 Annexe 2 : Cahier des charges

2.1 Introduction

L'apprentissage en robotique nécessite généralement un entraînement en conditions réelles chronophage et mobilisant du matériel spécialisé onéreux. La simulation constitue ainsi une alternative privilégiée. Toutefois, la création manuelle de scène virtuelles reste elle-même longue et peu scalable et limite la diversité et fréquence des expérimentations.

Ce projet, réalisé dans le cadre de l'UE PAI2D du Master 1 AI2D à Sorbonne Université, et encadré par le chercheur postdoctoral Emiland Garrabe et le Professeur Stéphane Doncieux de l'Institut des Systèmes Intelligents et de Robotique (ISIR), vise à automatiser la génération de scène de simulation robotique grâce aux modèles fondations. Ce dernier a pour ambition de réduire l'écart entre la simulation et la réalité.

Notre système sera capable de produire automatiquement des environnements simulables réalistes à partir de prompts textuels ou d'images, et prendra la forme d'un logiciel intermédiaire entre un Modèle Vision Language (VLM) et le moteur de simulation Genesis, chargé de traduire des descriptions sémantique ou visuelles en configurations exploitables.

2.2 Objectifs

L'objectif de ce projet est de développer un logiciel capable de générer automatiquement des scènes 3D simulables à partir d'une entrée textuelle (prompt) et/ou d'une image de référence.

Le système doit agir comme une couche intermédiaire entre des modèles fondations (LLM et VLM) et le moteur de simulation Genesis, en traduisant une description sémantique ou visuelle en une configuration exploitable par l'API Python du simulateur.

Plus précisément, le logiciel devra :

- Produire un fichier de configuration au format JSON contenant les informations nécessaires à la génération d'une scène 3D valide sous Genesis, à partir d'un prompt ou d'une image de référence analysés par les modèles fournis via Ollama (Llama 3.1 pour le texte et Llama 3.2 pour la vision).
- Instancier, simuler et tester automatiquement la scène générée dans Genesis
- Garantir la cohérence physique et la fidélité de la scène générée :
 - objets en lévitation, interpénétration, collision, etc.
 - respecter les relations spatiales et dimensions relatives
 - rester fidèle à la description en entrée
- Permettre potentiellement une extension future du système, notamment :
 - l'ajout ou la modification d'objets sur commande,
 - l'amélioration progressive de la précision géométrique,

Une phase de validation sur robot réel sera également possible sous réserve de temps disponible.

2.3 Description du système

Notre logiciel traduira des entrées en une scène de simulation robotique. Celles-ci pourront être : des prompts textuels, qui consistent en des indications sur la scène à configurer ("Crée une scène pour l'apprentissage de saisie de mug", ou encore "Mets un crayon à droite du mug sur la table" par exemple) ou des images de référence (photographie de la scène à reproduire). En entrée pourront aussi figurer des mesures physiques fournies par l'utilisateur. La sortie sera une scène générée sur Genesis qui correspondra aux instructions données.

Une bibliothèque d'objet URDF (Unified Robot Description Format) prédéfinie sera fournie par l'ISIR. Ce projet ne nécessite donc pas de générer les objets en eux-mêmes.

Le logiciel reposera donc sur une architecture modulaire en plusieurs blocs.

2.3.1 Module d'interprétation sémantique (LLM / VLM)

Ce premier module sera responsable de l'analyse de l'entrée de l'utilisateur :

Cas 1 : Prompt textuel. Dans le cas d'une entrée textuelle (exp : "Place un mug et un livre sur une table") le modèle Llama 3.1 sera chargé de :

- Identifier les objets présents dans la scène (mug, table, livre). Les objets seront limités à ceux présents dans la bibliothèque URDF.

- Extraire leurs relations spatiales (sur, sous, derrière, etc.)
- En déduire leur positions et dimensions relatives
- Générer une représentation structurée au format JSON, par exemple :

```
{
  "object_type": "mug",
  "x": 0.0,
  "y": 0.0,
  "z": 0.0,
  "quaternions": [],
}
```

Cas 2 : Image de référence. Dans le cas d’une image, nous exploiterons le modèle Llama 3.2-Vision dans le but de :

- Détecter les objets principaux de la scène.
- Déterminer la pertinence de ceux ci au problème (lorsque l’objectif est d’entraîner un bras robotique à agripper une tasse, la présence d’autres objets sur la table n’est pas toujours pertinente)
- Estimer leurs proportions relatives, leur position approximative et leurs relations spatiales.
- Générer une représentation structurée intermédiaire au format JSON

Dans ce mode, l’objectif est de générer une scène 3D la plus fidèle possible à l’image de référence. Pour se faire, un socle d’itération et de validation, détaillé dans une prochaine partie, sera mis en place.

Le module LLM/VLM produit ainsi un fichier JSON contenant le type des objets, leurs dimensions, leurs coordonnées 3D normalisées, leurs orientations (quaternions) et leurs relations structurelles relatives.

Cas 3 : Mesures réelles. Dans ce mode, des mesures physique réelles de la scène à reproduire viendront compléter un prompt ou une image. Ce module est utile dans le cas du besoin de placer un objet dans une position très précise.

2.3.2 Module de génération de scène Genesis

Ce module convertira une configuration JSON en instructions exploitables par l’API Genesis.

Il sera responsable de :

- Créer une scène vide dans l’environnement de simulation et charger les objets 3D identifiés en les associant aux assets de la bibliothèque URDF (Unified Robot Description Format) de l’ISIR
- Instancier chaque objet et appliquer positions et orientations.
- Vérifier la bonne initialisation des entités (collisions, autres contraintes éventuelle) et initier la simulation.

2.3.3 Boucle de validation et correction

La boucle de validation itérative permet d’éviter les comportements irréalistes et de vérifier le respect des lois de la physique. Les objets devront être décollés du sol, par exemple, pour pouvoir être saisis.

Dans le cas d’une entrée visuelle, cette boucle pourra effectuer les vérifications de validité physique et comparer le résultat à l’image de référence.]

Dans le cas d’une entrée textuelle, la partie validation via VLM aura principalement pour rôle de vérifier les inconsistances physique (une mauvaise position d’un objet, collision, interpénétration) et d’aider à la correction.

La boucle consiste à :

- Comparer la scène générée obtenue avec l’image de référence et identifier les divergences majeures (objets manquants, mauvaise disposition etc.), via VLM, et/ou vérifier le respect des lois physiques et la cohérence de la scène.
- Ajuster les paramètres de la scène en conséquence.
- Itérer jusqu’à atteindre un seuil de fidélité à définir.

Ce mécanisme itératif a pour ambition d’améliorer progressivement la fidélité structurelle entre l’image/le prompt et la scène simulée.

2.3.4 Interface utilisateur

Le système comprendra une interface minimale permettant de soumettre un prompt ou une image, ainsi qu'une visualisation de la scène générée.

2.3.5 Architecture globale

La pipeline générale du système est la suivante :

- Entrée utilisateur (texte ou image)
- Interprétation LLM/VLM
- Génération d'un JSON structuré
- Conversion vers API Genesis
- Génération de la scène simulable
- Boucle de validation itérative
- Export

Cette pipeline est également représentée par un flowchart dans la *Figure 1*.

2.4 Contraintes

Contraintes logicielles : Nous utilisons les modèles llama 3.1 et 3.2-vision avec Ollama pour l'automatisation de la génération de scène. Genesis (simulateur python) nous permet de générer les instances de scènes de simulation. Les objets pour la simulation viendront d'une bibliothèque d'objets URDF (Unified Robot Description Format) de l'ISIR.

Contraintes matérielles : La simulation Genesis et l'utilisation de LLM nécessitent des ressources GPU conséquentes. Le processus de simulation est donc lent sur certaines machines. Le bras Emika Franka de l'ISIR sera nécessaire dans le cas d'une validation sur robot réel.

2.5 Évaluation et critère d'acceptation

La phase de test comprendra la génération et reproduction de scènes. On donnera au logiciel une scène générée par le passé pour vérifier à quelle précision celui-ci pourra la reproduire. La génération devra également être relativement rapide. On regardera plusieurs critères :

Précision : un certain écart minimal entre la position réelle demandée et la simulation. La précision est surtout intéressante lorsque la simulation est basée sur des mesures réelles.

Validité/vraisemblance : la validité physique de la scène.

Taux de succès en 'one-shot' : la proportion de scènes valides (sans collision, objets correctement placés) sans intervention humaine.

Transférabilité, reproductibilité des résultats dans le monde réel (sim-to-real) : succès d'une tâche sur le bras Franka Emika, basée sur la scène générée.

2.6 Planning

2.6.1 Deadlines générales du projet AI2D

- **Fin du premier mois du projet :** rendu du cahier des charges.
- **18 mai :** rendu du rapport.
- **21-22 mai :** soutenance.

2.6.2 Notre organisation

- **26 février :** rendu du cahier des charges.
- **8 mars :** fonctionnement du simulateur à partir de prompts.
- **29 mars :** fonctionnement du simulateur à partir de prompts.
- **1er avril :** fin du développement logiciel.
- **12 avril :** fin des expérimentations.
- **1er mai :** première version du rendu.
- **5 mai :** version finale du rendu.
- **18 mai :** rendu du rapport.
- **21-22 mai :** soutenance.

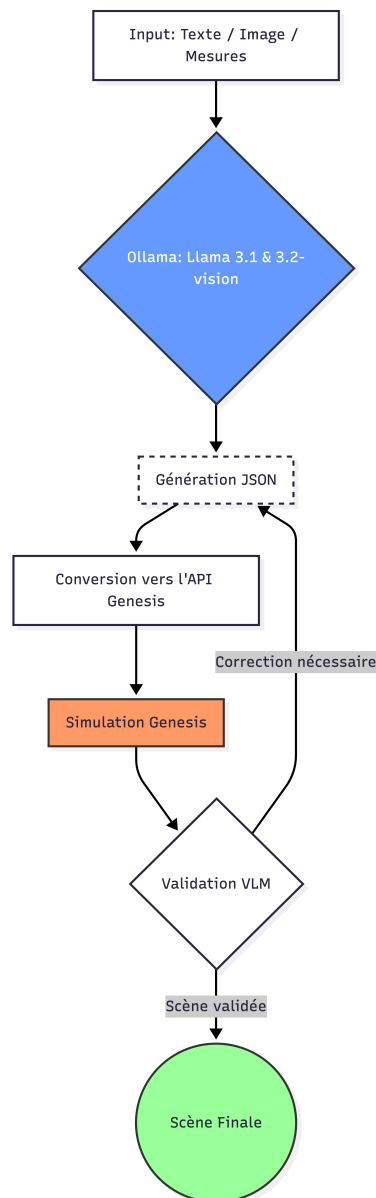


FIGURE 20 – Flowchart représentant la pipeline générale du système.

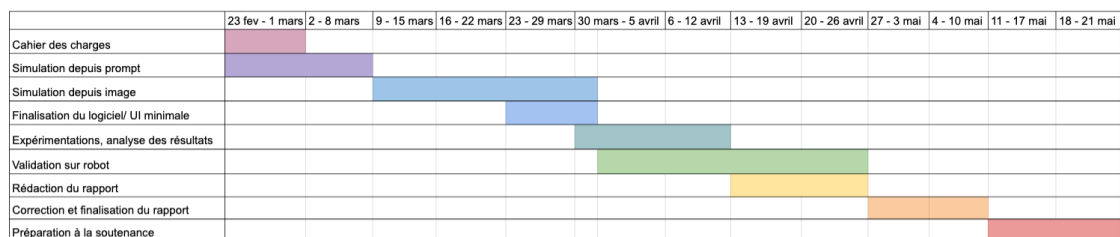


FIGURE 21 – Diagramme de Gantt représentant les étapes du projet dans les semaines du semestre